PROJECT3 (MATH214)

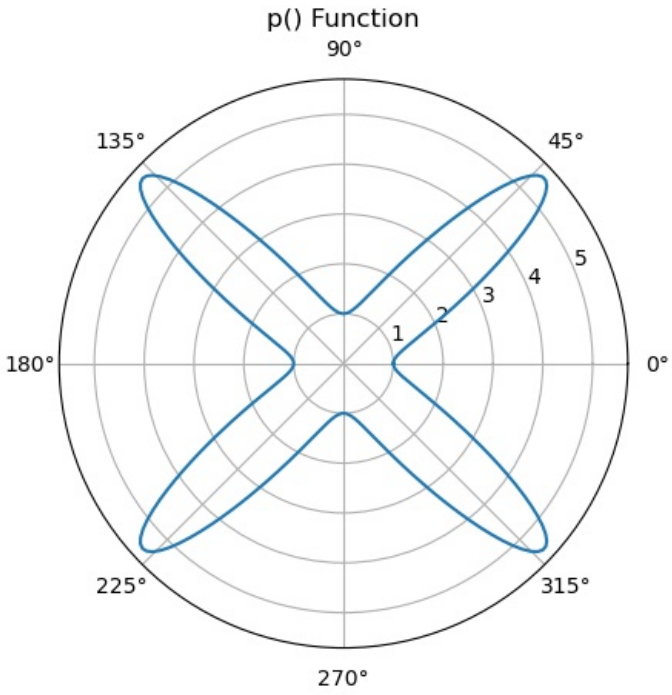
MUHAMMET ALİ AYVAZ

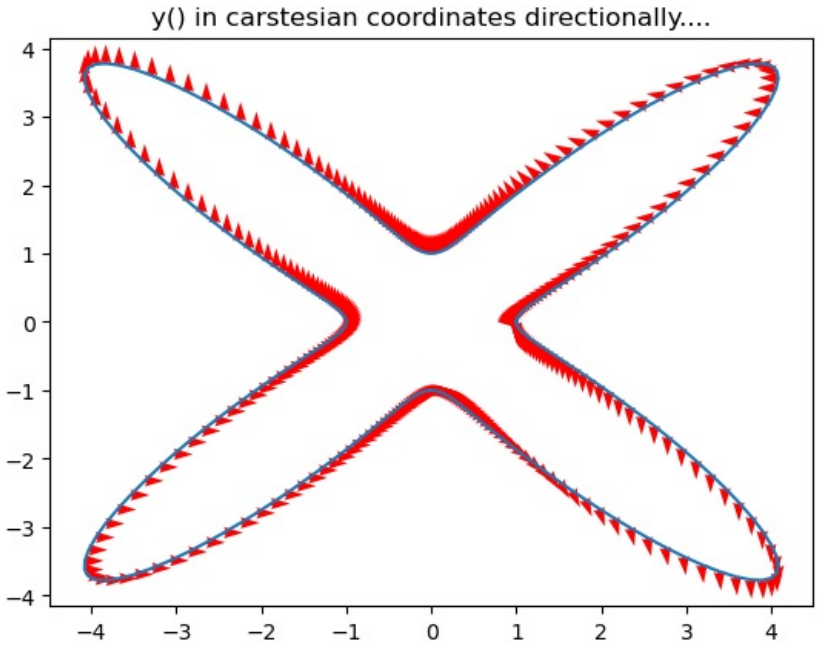
210102002078

In [10]:
```
import numpy as np  # LineSpace için
from matplotlib import pyplot as plt  # Grafik çizimleri için
import math  # Matematiksel işlemler için
```

A) In Section A, the required functions are provided, and the task involves plotting and displaying these functions in both polar and Cartesian coordinates using a computer. Additionally, the Cartesian representation should include orthogonal vectors. A detailed explanation of this process is provided below.

In [11]:
```
# Polar grafik fonksiyonu
def p(x):  # x radyan cinsinden. (Rapordaki p fonksiyonu)
    y1 = abs(math.cos(x))**8
    y2 = abs(math.sin(x))**6
    rval = (y1 + y2)**(-1)
    return rval

# x değerlerini oluşturuyoruz (0 ile 2π arasında 360 eşit aralık)
x_values = np.linspace(0, 2 * np.pi, 360)

# y değerlerini hesaplıyoruz
y_values = []
for i in range(0, len(x_values)):  # p fonksiyonuna göre y değerlerini ekliyoruz
    y_values.append(p(x_values[i]))

# Polar grafik oluşturma
plt.subplot(1, 1, 1, projection='polar')  # Polar grafik için
plt.plot(x_values, y_values)  # x ve y değerlerini çiz
plt.title("p() Function")  # Başlık
plt.show()  # Grafiği göster
#for cartesian graph....

def v_vector(x): # t_vector produces vertical vectors
    def dx_dx(x):
        y = -(3/4) * ((np.abs(np.cos(3*x/4)))**7) * np.sign(np.cos(3*x/4))
        return y
    def dy_dx(x):
        y = (3/4) * ((np.abs(np.sin(3*x/4)))**7) * np.sign(np.sin(3*x/4))
        return y
    dx_dx1 = dx_dx(x)
    dy_dx1 = dy_dx(x)
    l = np.sqrt(dx_dx1**2 + dy_dx1**2)
    ax = dx_dx1 / l
    ay = dy_dx1 / l
    return ax, ay


def calculate_coordinates(x_values, y_values): # the function in the report
    x_values_c = y_values * np.cos(x_values)
    y_values_c = y_values * np.sin(x_values)
    return x_values_c, y_values_c


t_vectors = []

for i in range(0,len(x_values)): # adding vertical vectors..
    t_vectors.append(v_vector(x_values[i]))

t_vector_x = []
t_vector_y = []

for i in range(0, len(t_vectors)):
    t_vector_x.append(t_vectors[i][0])
    t_vector_y.append(t_vectors[i][1])

plt.subplot(1, 1, 1) #visualizing for the graph
x_values_c, y_values_c = calculate_coordinates(x_values, y_values)
plt.plot(x_values_c, y_values_c, label='y()')

for i in range(len(x_values_c)):
    plt.quiver(x_values_c[i], y_values_c[i], t_vector_x[i], t_vector_y[i], color='red', scale=40)


plt.title('y() in carstesian coordinates directionally....')
plt.show()
```





B) In Section B, the objective is to calculate the derivatives (\frac{dx}{dp}), (\frac{dx}{dx}), and the function (L(x)) using the given functions (p(x)), (x), and (y). Additionally, plotting these derivatives and the function is required. This is achieved using the Forward Difference, Backward Difference, Three-Point Endpoint, and Three-Point Midpoint formulas applied separately. The finalized version of these tasks is provided below, with detailed explanations included as comment lines.

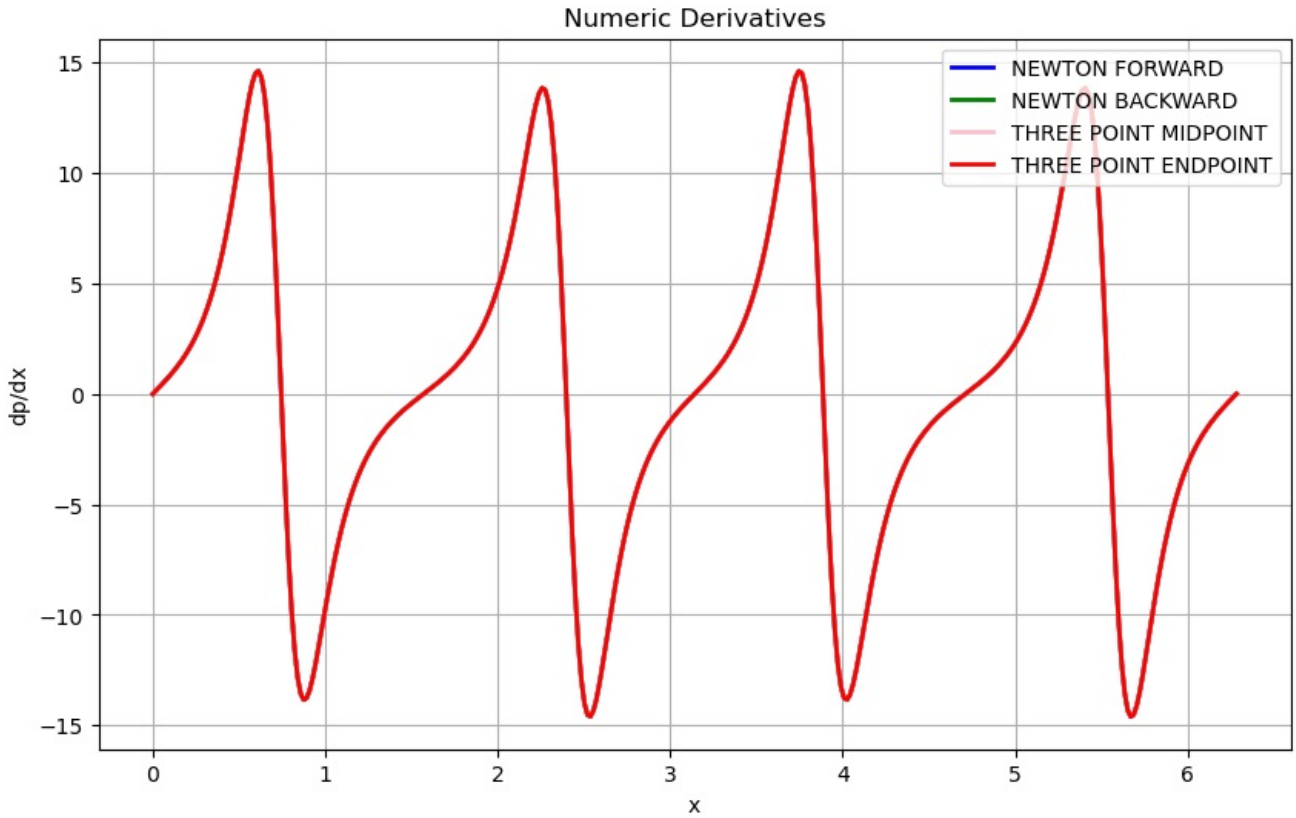
In [12]:
```
def newton_forward(f, x, h): #newton forward formula
    der_val = (f(x + h) - f(x)) / h
    return der_val

def newton_backward(f, x, h): #newton backward formula
    der_val = (f(x) - f(x - h)) / h
    return der_val

def three_point_midpoint(f, x, h): #three point midpoint formula
    der_val = (f(x + h) - f(x - h)) / (2 * h)
    return der_val

def three_point_endpoint(f, x, h): #three point endpoint formula
    der_val = (-3 * f(x) + 4 * f(x + h) - f(x + 2 * h)) / (2 * h)
    return der_val

h= 0.0000000001  #smal h value (to approximate)

# fiind_numeric_and_show function finds df/dx according to newton forward, backward... and function which is se
#if true ==> return that y_values, if not only shows the graph.The reason for using x is for seciton d
def find_numeric_and_show(function, y_label, label = False):

    df_dx_values1 = []
    df_dx_values2 = []
    df_dx_values3 = []
```

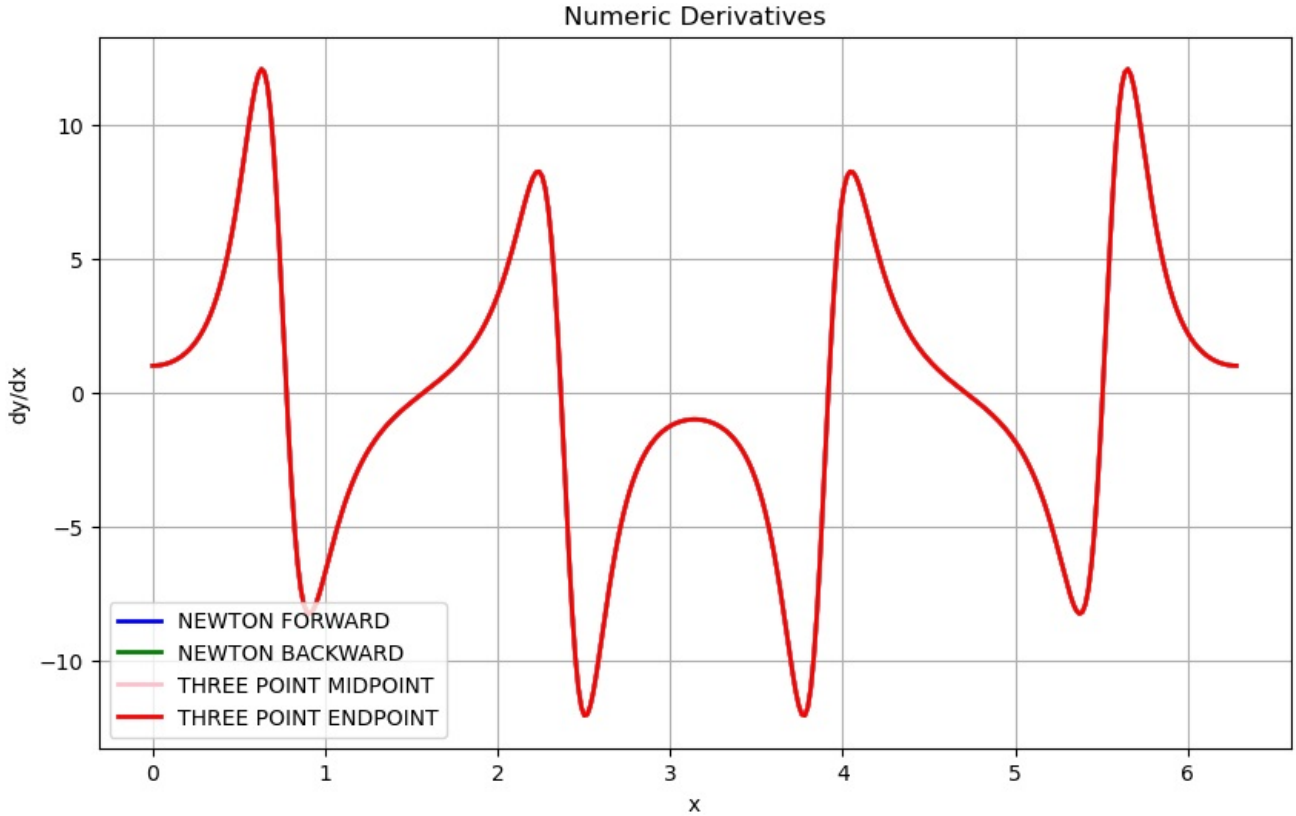
```
    df_dx_values4 = []
    for i in range(0, len(x_values)): #applying methods according to the sent function...
        df_dx_values1.append(newton_forward(function, x_values[i], h))
        df_dx_values2.append(newton_backward(function, x_values[i], h))
        df_dx_values3.append(three_point_midpoint(function, x_values[i], h))
        df_dx_values4.append(three_point_endpoint(function, x_values[i], h))

    if (label):
        return df_dx_values3
    #create the graph
    plt.figure(figsize=(10, 6))

    plt.plot(x_values, df_dx_values1, color="blue", linewidth=2, label="NEWTON FORWARD" )
    plt.plot(x_values, df_dx_values2, color="green", linewidth=2, label="NEWTON BACKWARD" )
    plt.plot(x_values, df_dx_values3, color="pink", linewidth=2, label="THREE POINT MIDPOINT" )
    plt.plot(x_values, df_dx_values4, color="red", linewidth=2, label="THREE POINT ENDPOINT" )

    plt.title('Numeric Derivatives')
    plt.xlabel('x')
    plt.ylabel(y_label)
    plt.legend()
    plt.grid(True)
    plt.show()

#creating desired functions ...
```

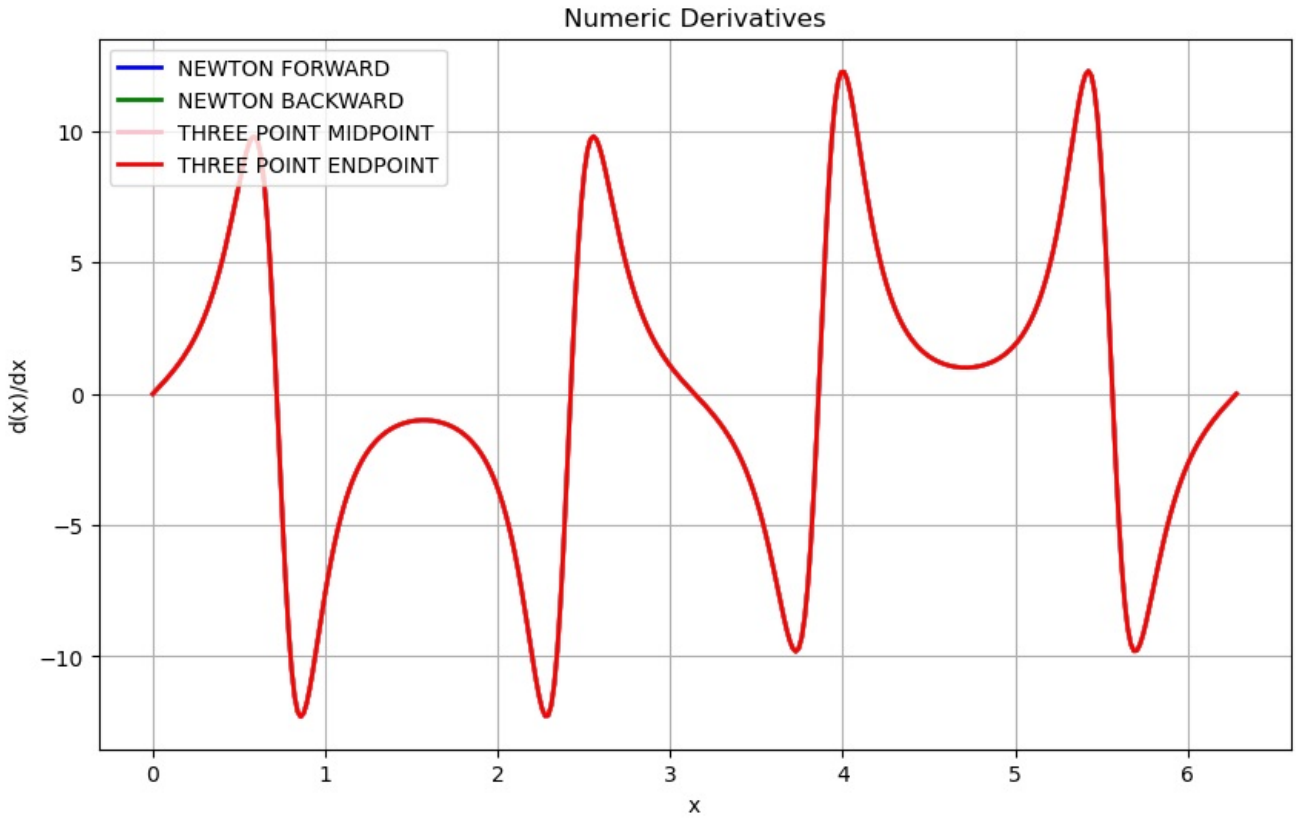
The functions dp/dx, dy/dx, and L(x) obtained using the forward difference, backward difference, three-point endpoint, three-point midpoint, and three-point central difference formulas are overlaid in the graph below.

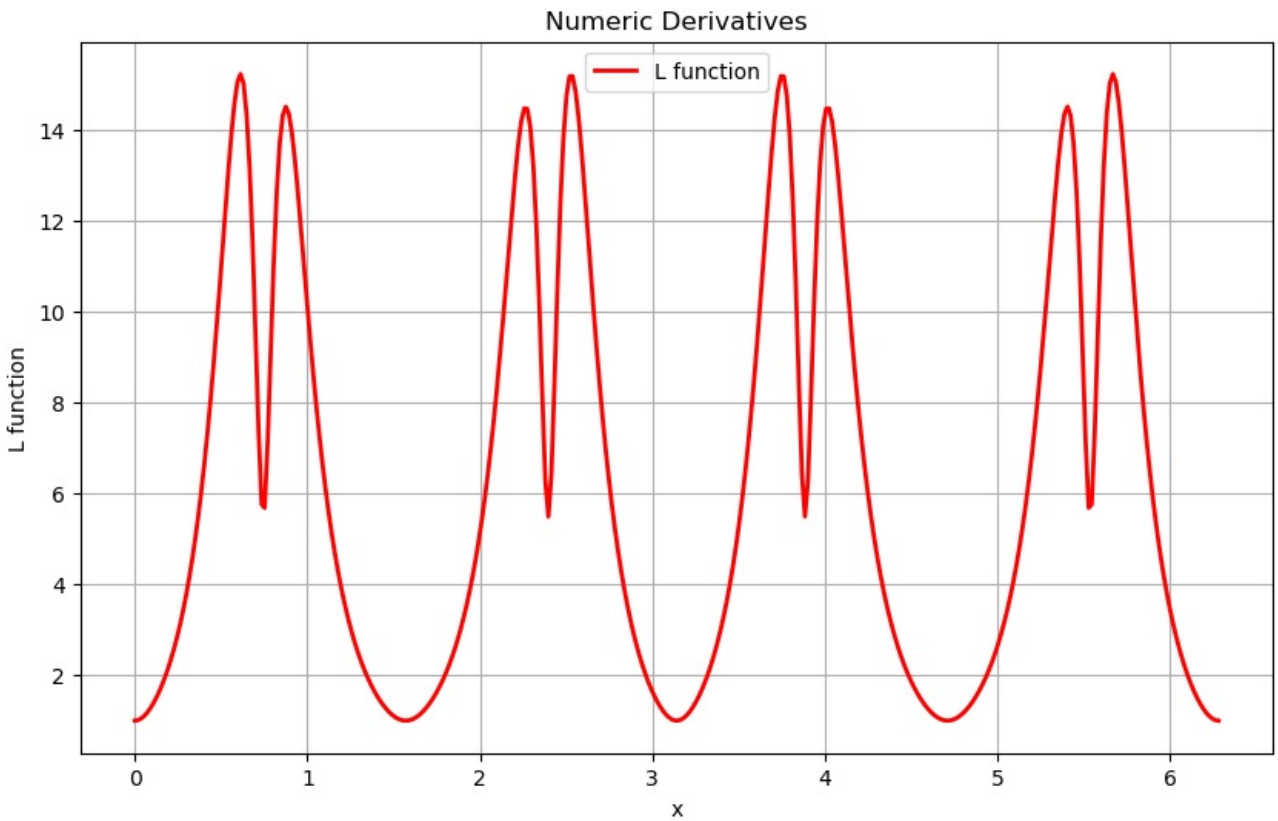
In [13]:

```
#for dp/dx:
find_numeric_and_show(p, "dp/dx")
dp_dx_list = find_numeric_and_show(p, "empty", True) #for using d section...
#for dy/dx:
def y_function(x):
    y = p(x) * np.sin(x)
    return y
find_numeric_and_show(y_function, "dy/dx")
dy_dx_list = find_numeric_and_show(y_function, "empty", True) #for using (d) section...
#for d(x)/dx
def x_function(x):
    x = p(x) * np.cos(x)
    return x
find_numeric_and_show(x_function, "d(x)/dx")
dx_dx_list = find_numeric_and_show(x_function, "empty", True)
#for L(x):
dx_x = find_numeric_and_show(x_function, "empty", True)
dy_dx = find_numeric_and_show(y_function, "empty", True)
sum_list = []
for i in range(0, len(x_values)):
    sum_list.append(np.sqrt(dx_x[i]**2 + dy_dx[i]**2))

#show l(x) function.

plt.figure(figsize=(10, 6))
plt.plot(x_values, sum_list, color="red", linewidth=2, label="L function" )
plt.title('Numeric Derivatives')
plt.xlabel('x')
plt.ylabel("L function")
plt.legend()
plt.grid(True)
plt.show()
```


Numeric Derivatives
NEWTON FORWARD
NEWTON BACKWARD
THREE POINT MIDPOINT
THREE POINT ENDPOINT
dp/dx
x
15
10
5
0
−5
−10
−15
0
1
2
3
4
5
6


Numeric Derivatives
NEWTON FORWARD
NEWTON BACKWARD
THREE POINT MIDPOINT
THREE POINT ENDPOINT
dy/dx
x
10
5
0
−5
−10
0
1
2
3
4
5
6





As seen, the functions obtained with the four methods overlap, indicating that there is not much difference between the four methods.

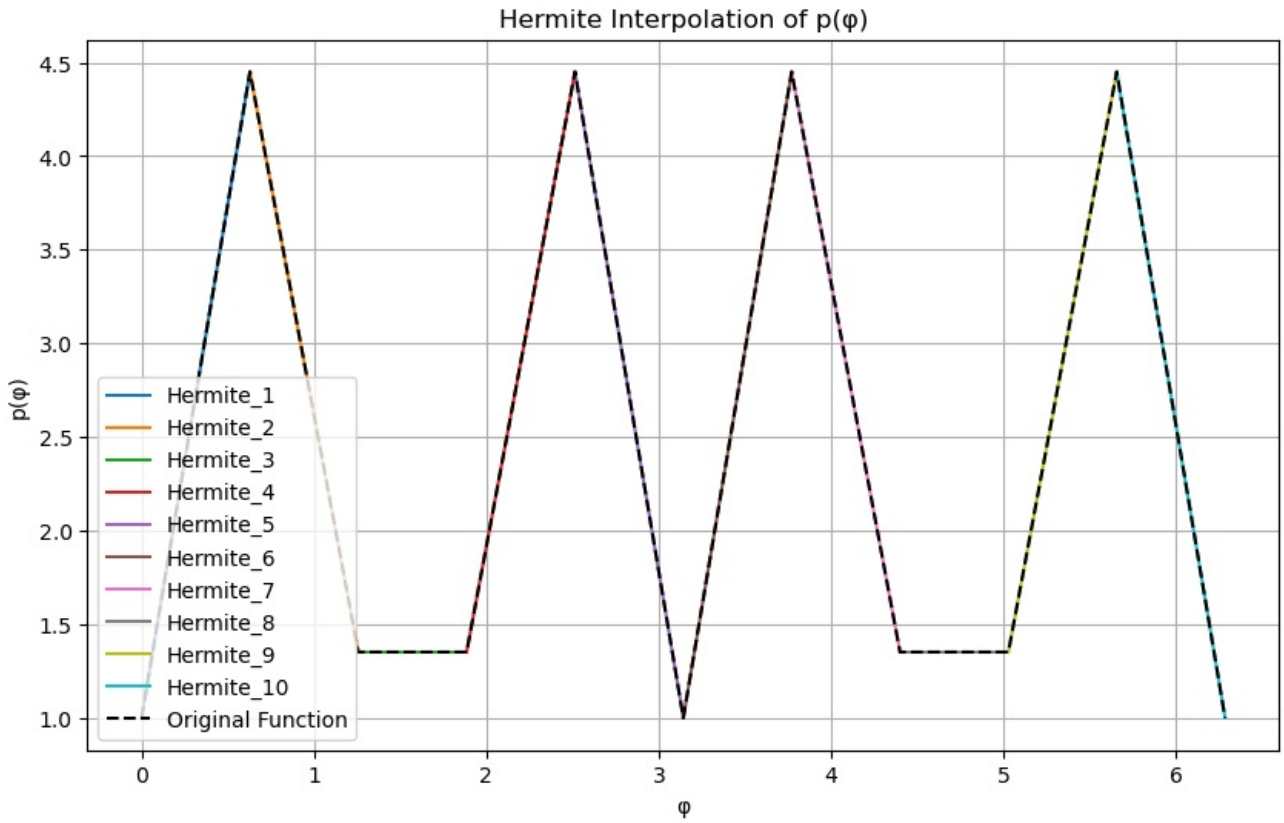
C) In Section C, it is stated that the Hermite function has been determined, but only the y-values of the Hermite function were calculated. However, the task requires finding the (p(x)) function. To address this, the coefficients of the Hermite function were calculated piecewise and printed. Subsequently, with the assistance of AI (ChatGPT), these printed piecewise functions were transformed into an array and written in functional form. As a result, we now have the piecewise Hermite(x) function, which approximates the values of (p(x)).

(Note: ChatGPT was used because the functions are too complex and lengthy for manual input.)

In [14]:
```
try_list = []

#Calculating 3rd-degree Hermite interpolation
def hermite_interpolation(x_start, x_end, y_start, y_end, slope_start, slope_end, x_value):
    t = (x_value - x_start) / (x_end - x_start)
    h00 = (1 + 2 * t) * (1 - t)**2
    h10 = t * (1 - t)**2
    h01 = t**2 * (3 - 2 * t)
```

```
    h11 = t**2 * (t - 1)

    if y_start not in try_list: #for observing section d , to collect form of formula
        try_list.append(y_start)
        print("x = {}, {} *(1 + 2 * t) * (1 - t)**2, {}* t * (1 - t)**2, {}* t**2 * (3 - 2 * t), {}* t**2 * (t
    return h00 * y_start + h10 * (x_end - x_start) * slope_start + h01 * y_end + h11 * (x_end - x_start) * slop


N = 10 # subintervals N
x_division = np.linspace(0, 2 * np.pi, N + 1)

plt.figure(figsize=(10, 6))

# Calculating the required values for Hermite interpolation while iterating over subintervals....
for i in range(N):
    x0, x1 = x_division[i], x_division[i + 1]
    p0, p1 = p(x0), p(x1)
    dp0, dp1 = (p(x1) - p(x0)) / (x1 - x0), (p(x1) - p(x0)) / (x1 - x0)

    x_vals = np.linspace(x0, x1, 100)
    # Calculating Hermite interpolation at each point...
    p_vals = []
    for x in x_vals:
        p_vals.append(hermite_interpolation(x0, x1, p0, p1, dp0, dp1, x))

    # shows each  Hermite interpolation....
    plt.plot(x_vals, p_vals, label='Hermite_{}'.format(i + 1))

real_values = []
for x in x_division:
    real_values.append(p(x))

plt.plot(x_division, real_values, label='Original Function', linestyle='--', color='black')

# for graph
plt.title('Hermite Interpolation of p(φ)')
plt.xlabel('φ')
plt.ylabel('p(φ)')
plt.legend()
plt.grid(True)
plt.show()
```

```
x = 0.0, 1.0 *(1 + 2 * t) * (1 - t)**2, 3.44938862178501* t * (1 - t)**2, 4.44938862178501* t**2 * (3 - 2 * t),
3.44938862178501* t**2 * (t - 1)
x = 0.6283185307179586, 4.44938862178501 *(1 + 2 * t) * (1 - t)**2, -3.098208488409944* t * (1 - t)**2, 1.35118
0133375066* t**2 * (3 - 2 * t), -3.098208488409944* t**2 * (t - 1)
x = 1.2566370614359172, 1.351180133375066 *(1 + 2 * t) * (1 - t)**2, -1.1102230246251565e-15* t * (1 - t)**2, 1
.3511801333750648* t**2 * (3 - 2 * t), -1.1102230246251565e-15* t**2 * (t - 1)
x = 1.8849555921538759, 1.3511801333750648 *(1 + 2 * t) * (1 - t)**2, 3.0982084884099472* t * (1 - t)**2, 4.449
388621785013* t**2 * (3 - 2 * t), 3.0982084884099472* t**2 * (t - 1)
x = 2.5132741228718345, 4.449388621785013 *(1 + 2 * t) * (1 - t)**2, -3.4493886217850127* t * (1 - t)**2, 1.0*
t**2 * (3 - 2 * t), -3.4493886217850127* t**2 * (t - 1)
x = 3.7699111843077517, 4.449388621785012 *(1 + 2 * t) * (1 - t)**2, -3.098208488409946* t * (1 - t)**2, 1.3511
80133375066* t**2 * (3 - 2 * t), -3.098208488409946* t**2 * (t - 1)
```

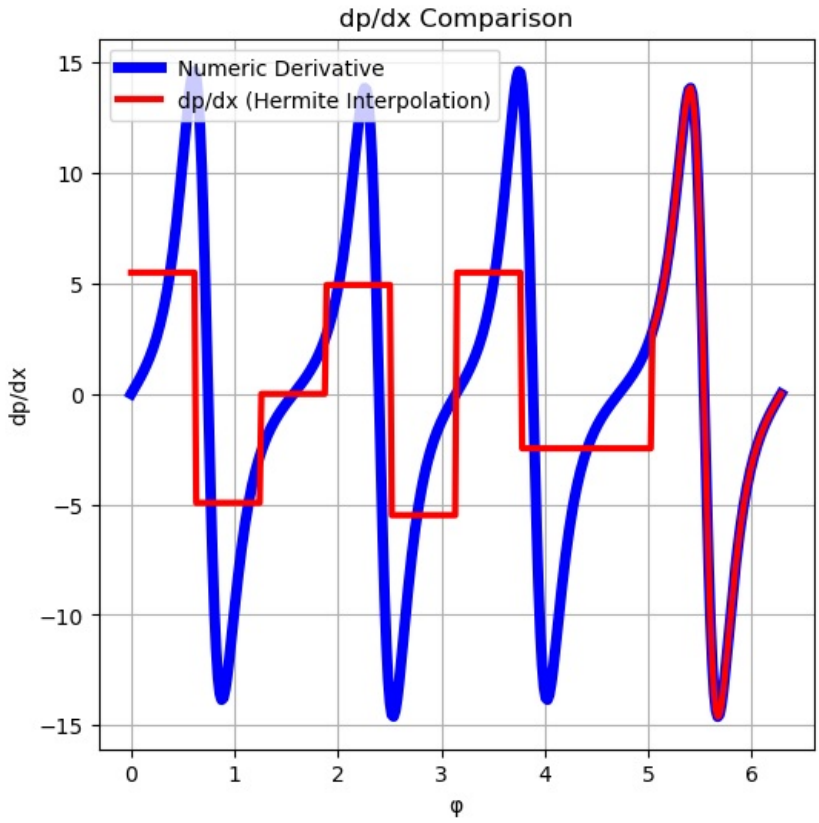


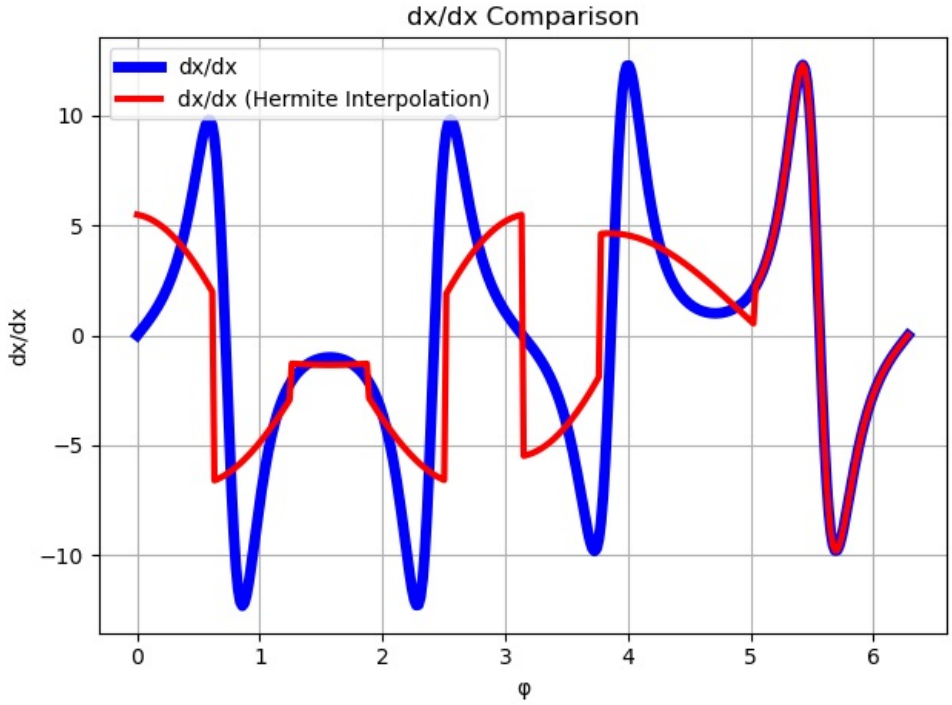
Based on the provided graph, the piecewise Hermite functions closely align with the original (p(x)) function, as evidenced by the overlapping lines. This indicates a high level of accuracy and no noticeable error between the Hermite interpolation and the actual (p(x)) values. When experimenting with different values of (n) (the number of interpolation points), it can be observed that as (n) decreases, the approximation diverges more significantly from the original function. This demonstrates that higher (n) values result in better interpolation accuracy, while lower (n) values reduce the precision of the Hermite approximation.

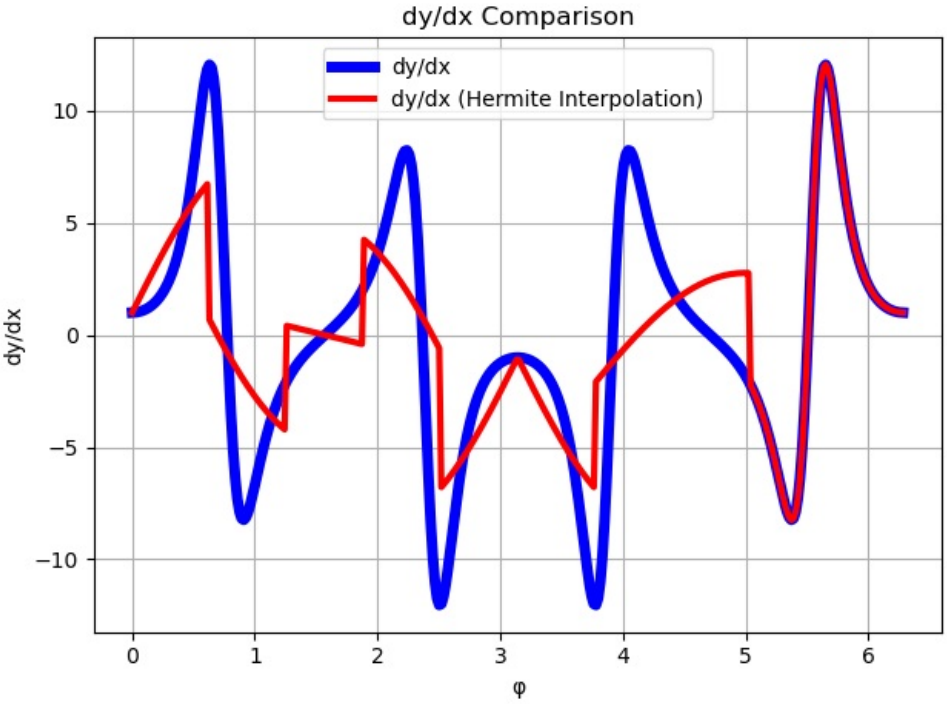
D) In Section D, the task is to rediscover dp/dx, dy/dx, dx/dx, and L(x) functions using the obtained Hermite function. These functions, along with the actual p(x) functions, are overlaid, and the relative error is determined

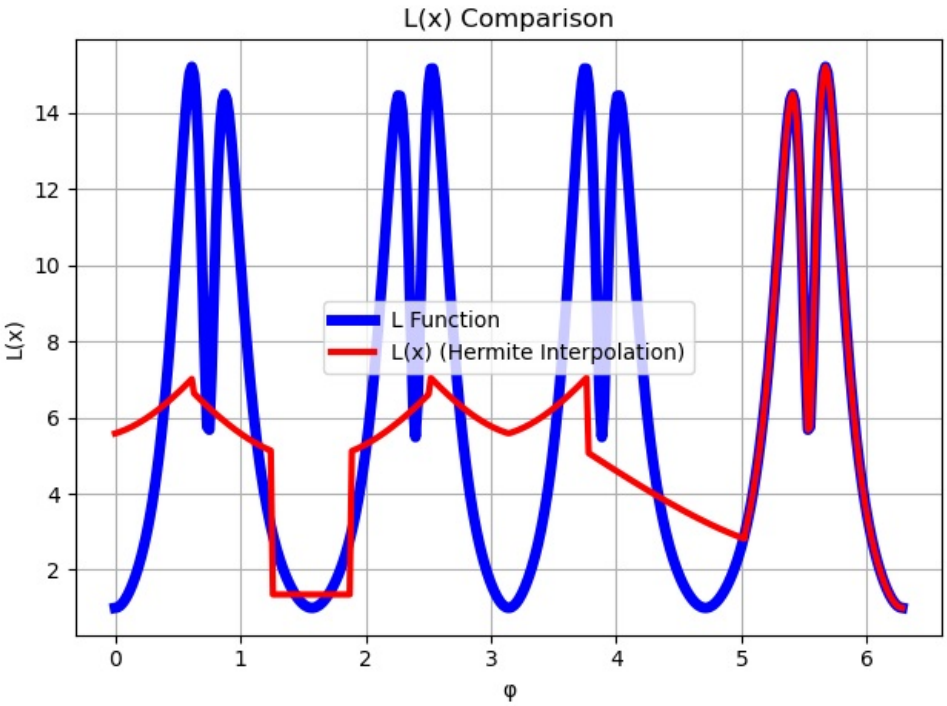
In [15]:
```
# the printed outputs were copied and transformed into an array in AI. Later, the piecewise_hermit_f function w

def piecewise_hermit_f(x):
    x_division = [0, 0.6283185307179586, 1.2566370614359172, 1.8849555921538759, 2.5132741228718345, 3.14159265

    for i in range(len(x_division) - 1):
        x0, x1 = x_division[i], x_division[i + 1]
        if x0 <= x <= x1:
            p0, p1 = p(x0), p(x1)
            dp0, dp1 = (p(x1) - p(x0)) / (x1 - x0), (p(x1) - p(x0)) / (x1 - x0)
            t = (x - x0) / (x1 - x0)
            h00 = (1 + 2 * t) * (1 - t)**2
            h10 = t * (1 - t)**2
            h01 = t**2 * (3 - 2 * t)
            h11 = t**2 * (t - 1)
            return h00 * p0 + h10 * (x1 - x0) * dp0 + h01 * p1 + h11 * (x1 - x0) * dp1

    return p(x)  # Default: return p(x) if x is outside the specified range

def x_function_H(x): #it is created to obtain x function of hermit
    return piecewise_hermit_f(x) * np.cos(x)
def y_function_H(x): #it is created to obtain y function of hermit
    return piecewise_hermit_f(x) * np.sin(x)
def calculate_derivative(f, x): # calculating a derivative of any function with name of the function....
    h = 1e-10
    return (f(x + h) - f(x)) / h
def L_function_H(x): #it is created to obtain l(x) function....
    return (calculate_derivative(x_function_H, (x))**2 + calculate_derivative(y_function_H, (x))**2)**0.5

#obtaining the functions by using hermit formula
#for dp/dx:
y_val_dp_dx = []
for i in range(0, len(x_values)):
    y_val_dp_dx.append(calculate_derivative(piecewise_hermit_f, x_values[i]))
#for dx_dx:
y_val_dx_dx = []
for i in range(0, len(x_values)):
    y_val_dx_dx.append(calculate_derivative(x_function_H, x_values[i]))
#for dy_dx:
y_val_dy_dx = []
for i in range(0, len(x_values)):
    y_val_dy_dx.append(calculate_derivative(y_function_H, x_values[i]))
#for L function :
y_val_L =[]
for i in range(0, len(x_values)):
    y_val_L.append(L_function_H(x_values[i]))

#shows graphs :

plt.figure(figsize=(6, 6))
plt.plot(x_values, dp_dx_list, label='Numeric Derivative',  color="blue", linewidth=5)
plt.title('Numeric Derivative of p(φ)')
plt.xlabel('φ')
plt.ylabel('dp/dx')
plt.legend()
plt.grid(True)
plt.plot(x_values, y_val_dp_dx, label='dp/dx (Hermite Interpolation)',  color="red", linewidth=3)
plt.title('dp/dx Comparison')
plt.xlabel('φ')
plt.ylabel('dp/dx')
plt.legend()
plt.grid(True)
plt.show()

#****************************************

plt.plot(x_values, dx_dx_list, label='dx/dx',  color="blue", linewidth=5)
plt.title('dx/dx')
plt.xlabel('φ')
plt.ylabel('dx/dx')
plt.legend()
plt.grid(True)
plt.plot(x_values, y_val_dx_dx, label='dx/dx (Hermite Interpolation)',  color="red", linewidth=3)
plt.title('dx/dx Comparison')
plt.xlabel('φ')
plt.ylabel('dx/dx')
plt.legend()
```

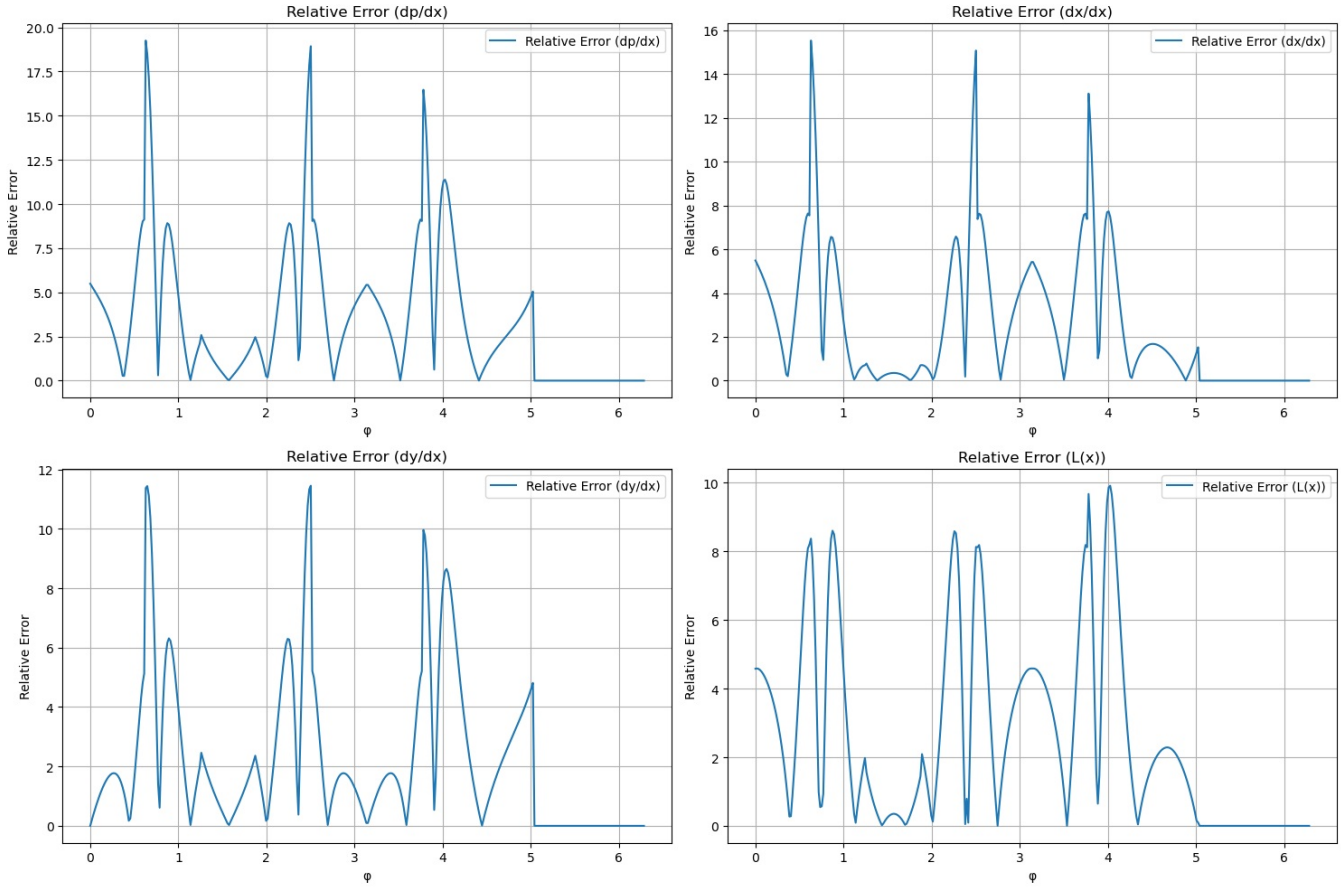
```
plt.grid(True)
plt.tight_layout()
plt.show()
#****************************************

plt.plot(x_values, dy_dx_list, label='dy/dx',  color="blue", linewidth=5)
plt.title('dy/dx')
plt.xlabel('φ')
plt.ylabel('dy/dx')
plt.legend()
plt.grid(True)
plt.plot(x_values, y_val_dy_dx, label='dy/dx (Hermite Interpolation)',  color="red", linewidth=3)
plt.title('dy/dx Comparison')
plt.xlabel('φ')
plt.ylabel('dy/dx')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#****************************************

plt.plot(x_values, sum_list, label='L Function',  color="blue", linewidth=5)
plt.title('L(x)')
plt.xlabel('φ')
plt.ylabel('L(x)')
plt.legend()
plt.grid(True)
plt.plot(x_values, y_val_L, label='L(x) (Hermite Interpolation)',  color="red", linewidth=3)
plt.title('L(x) Comparison')
plt.xlabel('φ')
plt.ylabel('L(x)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()



# calculating relative errros....

r_e_dp_dx = np.abs(np.array(dp_dx_list) - np.array(y_val_dp_dx))
r_e_dx_dx = np.abs(np.array(dx_dx_list) - np.array(y_val_dx_dx))
r_e_dy_dx = np.abs(np.array(dy_dx_list) - np.array(y_val_dy_dx))
r_e_L = np.abs(np.array(sum_list) - np.array(y_val_L))

# showing graphs(relative errors)...

plt.figure(figsize=(15, 10))

# for dp/dx

plt.subplot(2, 2, 1)
plt.plot(x_values, r_e_dp_dx, label='Relative Error (dp/dx)')
plt.title('Relative Error (dp/dx)')
plt.xlabel('φ')
plt.ylabel('Relative Error')
plt.legend()
plt.grid(True)

# for dx/dx
plt.subplot(2, 2, 2)
plt.plot(x_values, r_e_dx_dx, label='Relative Error (dx/dx)')
plt.title('Relative Error (dx/dx)')
plt.xlabel('φ')
plt.ylabel('Relative Error')
plt.legend()
plt.grid(True)

# for dy/dx
plt.subplot(2, 2, 3)
plt.plot(x_values, r_e_dy_dx, label='Relative Error (dy/dx)')
plt.title('Relative Error (dy/dx)')
plt.xlabel('φ')
plt.ylabel('Relative Error')
plt.legend()
plt.grid(True)

# for L(x)
plt.subplot(2, 2, 4)
plt.plot(x_values, r_e_L, label='Relative Error (L(x))')
plt.title('Relative Error (L(x))')
plt.xlabel('φ')
plt.ylabel('Relative Error')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```


dp/dx Comparison
Numeric Derivative
dp/dx (Hermite Interpolation)
dp/dx
φ
15
10
5
0
−5
−10
−15
0
1
2
3
4
5
6


dx/dx Comparison
dx/dx
dx/dx (Hermite Interpolation)
dx/dx
φ
10
5
0
−5
−10
0
1
2
3
4
5
6


dy/dx Comparison
dy/dx
dy/dx (Hermite Interpolation)
dy/dx
φ
10
5
0
−5
−10
0
1
2
3
4
5
6


L(x) Comparison
L Function
L(x) (Hermite Interpolation)
L(x)
φ
14
12
10
8
6
4
2
0
1
2
3
4
5
6



E-F) In Section e and f, it is requested to find all functions, previously obtained using the Hermite method in Section D, with the cubic spline method instead. However, this is a more challenging problem than Hermite because the boundaries of piecewise functions have extremely large slope values for both the function and its derivative. It has been determined through trial and error that dividing the interval [0, 2π] into 85 segments, rather than 360, is the most optimized approach. Additionally, various online research has been conducted, and it has been identified that the tridiagonal matrix algorithm is one of the best methods for solving this problem. The necessary references have been added to the end of the report. This algorithm has been adapted and edited with ChatGPT, resulting in the creation of the derivative_cubic(x) and cubic_formula functions. Subsequently, graphs along with error rates have been presented

In [16]:
```
# Cubic spline interpolation function
def cubic_spline_interpolation(x, y, x_interp):
    # Get the number of data points
    n = len(x)

    # Calculate the distances h between data points
    h = np.diff(x)

    # Calculate the differences in y values between data points
    delta_y = np.diff(y)

    # Calculate the derivative alpha for each h distance
    alpha = delta_y / h

    # Allocate space for the lower triangle of the tridiagonal matrix
    l = np.zeros(n)
    mu = np.zeros(n)
    z = np.zeros(n)

    # Manually set the first element
    l[1] = 3 * (alpha[1] - alpha[0]) / (h[1] + h[0])

    # Calculate the remaining elements with a for loop
    for i in range(2, n-1):
        l[i] = 3 * (alpha[i] - alpha[i-1]) / (h[i] + h[i-1])
        mu[i] = h[i-1] / (h[i-1] + h[i] - h[i-1] * mu[i-1])
        z[i] = (l[i] - h[i-1] * z[i-1]) / (h[i-1] + h[i] - h[i-1] * mu[i-1])

    # Calculate spline coefficients
    l_c_e = np.zeros(n)
    c_c_e = np.zeros(n)
    d_c_e = np.zeros(n)

    for j in range(n-2, 0, -1):
        c_c_e[j] = z[j] - mu[j] * c_c_e[j+1]
        l_c_e[j] = alpha[j] - (alpha[j-1] - l_c_e[j-1]) * mu[j]
        d_c_e[j] = (c_c_e[j+1] - c_c_e[j]) / (3 * h[j])

    # Calculate the b coefficients
    b_c_e = alpha - h * (c_c_e[1:] + 2 * c_c_e[:-1]) / 3
```

```python
    interpolated_values = np.zeros_like(x_interp)

    # Perform interpolation for each x_interp value
    for i in range(len(x_interp)):
        # Determine the interval in which x_interp lies
        index = np.searchsorted(x, x_interp[i])
        index = np.clip(index - 1, 0, n-2).astype(int)

        # Calculate the first, second, third, and fourth-degree terms based on the x value in the respective in
        dx = x_interp[i] - x[index]
        interpolated_values[i] = y[index] + b_c_e[index] * dx + c_c_e[index] * dx**2 + d_c_e[index] * dx**3

    # Print the cubic spline interpolation function
    derivative_values = np.zeros_like(x_interpolation_points)

    for i in range(len(x_interpolation_points)):
        # Determine the interval in which x_interp lies
        index = np.searchsorted(x_values, x_interpolation_points[i])
        index = np.clip(index - 1, 0, len(x_values)-2).astype(int)

        # Calculate the derivative using the coefficients of the cubic spline in the respective interval
        dx = x_interpolation_points[i] - x_values[index]
        derivative_values[i] = b_c_e[index] + 2 * c_c_e[index] * dx + 3 * d_c_e[index] * dx**2

    # Prints the derivative values to collect values to obtain the formula....
    for j in range(len(x_values)-1):
        print(f"S'{j}(x) = {derivative_values[j]}  for  {x_values[j]} <= x <= {x_values[j+1]}")

    for j in range(n-1): # prints the y values to collect to obtain the formula...
        print(f"S{j}(x) = {y[j]} + {b_c_e[j]} * (x - {x[j]}) + {c_c_e[j]} * (x - {x[j]})^2 + {d_c_e[j]} * (x -

    return interpolated_values

# that derivative_cubic function is obtained with chatgpt to do array.
# in 443.row, it is printed
def derivative_cubic(x):
    coefficients = [
        (0.0, 0.07479982508547127, 0.1532),
        (0.07479982508547127, 0.14959965017094254, 0.1587),
        (0.14959965017094254, 0.2243994752564138, 0.1641),
        (0.2243994752564138, 0.2991993003418851, 0.1696),
        (0.2991993003418851, 0.37399912542735636, 0.1750),
        (0.37399912542735636, 0.4487989505128276, 0.1805),
        (0.4487989505128276, 0.5235987755982989, 0.1859),
        (0.5235987755982989, 0.5983986006837702, 0.1923),
        (0.5983986006837702, 0.6731984257692414, 0.1987),
        (0.6731984257692414, 0.7479982508547127, 0.2052),
        (0.7479982508547127, 0.822798075940184, 0.2116),
        (0.822798075940184, 0.8975979010256552, 0.2179),
        (0.8975979010256552, 0.9723977261111265, 0.2234),
        (0.9723977261111265, 1.0471975511965979, 0.2290),
        (1.0471975511965979, 1.121997376282069, 0.2345),
        (1.121997376282069, 1.1967972013675403, 0.2401),
        (1.1967972013675403, 1.2715970264530116, 0.2457),
        (1.2715970264530116, 1.3463968515384828, 0.2512),
        (1.3463968515384828, 1.421196676623954, 0.2568),
        (1.421196676623954, 1.4959965017094254, 0.2623),
        (1.4959965017094254, 1.5707963267948966, 0.2679),
        (1.5707963267948966, 1.645596151880368, 0.2734),
        (1.645596151880368, 1.7203959769658392, 0.2789),
        (1.7203959769658392, 1.7951958020513104, 0.2845),
        (1.7951958020513104, 1.8699956271367817, 0.2900),
        (1.8699956271367817, 1.944795452222253, 0.2956),
        (1.944795452222253, 2.019595277307724, 0.3011),
        (2.019595277307724, 2.0943951023931957, 0.3067),
        (2.0943951023931957, 2.169194927478667, 0.3122),
        (2.169194927478667, 2.243994752564138, 0.3178),
        (2.243994752564138, 2.3187945776496095, 0.3233),
        (2.3187945776496095, 2.3935944027350806, 0.3289),
        (2.3935944027350806, 2.4683942278205517, 0.3344),
        (2.4683942278205517, 2.5431940529060233, 0.3399),
        (2.5431940529060233, 2.6179938779914944, 0.3455),
        (2.6179938779914944, 2.6927937030769655, 0.3510),
        (2.6927937030769655, 2.767593528162437, 0.3566),
        (2.767593528162437, 2.842393353247908, 0.3621),
        (2.842393353247908, 2.9171931783333793, 0.3677),
        (2.9171931783333793, 2.991993003418851, 0.3732),
        (2.9171931783333793, 2.991993003418851, 0.3732),
        (2.991993003418851, 3.066792828504322, 0.3788),
        (3.066792828504322, 3.141592653589793, 0.3843),
        (3.141592653589793, 3.2163924786752647, 0.3899),
        (3.2163924786752647, 3.291192303760736, 0.3954),
        (3.291192303760736, 3.365992128846207, 0.4010),
        (3.365992128846207, 3.4407919539316785, 0.4065),
        (3.4407919539316785, 3.5155917790171496, 0.4121),
        (3.5155917790171496, 3.5903916041026207, 0.4176),
        (3.5903916041026207, 3.6651914291880923, 0.4232),
        (3.6651914291880923, 3.7399912542735634, 0.4287),
```

```
        (3.7399912542735634, 3.814791079359035, 0.4343),
        (3.814791079359035, 3.889590904444506, 0.4398),
        (3.889590904444506, 3.964390729529977, 0.4454),
        (3.964390729529977, 4.039190554615448, 0.4510),
        (4.039190554615448, 4.11399037970092, 0.4565),
        (4.11399037970092, 4.188790204786391, 0.4621),
        (4.188790204786391, 4.263590029871862, 0.4676),
        (4.263590029871862, 4.338389854957334, 0.4732),
        (4.338389854957334, 4.413189680042805, 0.4787),
        (4.413189680042805, 4.487989505128276, 0.4843),
        (4.487989505128276, 4.562789330213747, 0.4898),
        (4.562789330213747, 4.637589155299219, 0.4954),
        (4.637589155299219, 4.71238898038469, 0.5010),
        (4.71238898038469, 4.787188805470161, 0.5065),
        (4.787188805470161, 4.861988630555633, 0.5121),
        (4.861988630555633, 4.9367884556411035, 0.5176),
        (4.9367884556411035, 5.011588280726575, 0.5232),
        (5.011588280726575, 5.086388105812047, 0.5287),
        (5.086388105812047, 5.161187930897517, 0.5343),
        (5.161187930897517, 5.235987755982989, 0.5398),
        (5.235987755982989, 5.31078758106846, 0.5454),
        (5.31078758106846, 5.385587406153931, 0.5510),
        (5.385587406153931, 5.460387231239403, 0.5565),
        (5.460387231239403, 5.535187056324874, 0.5621),
        (5.535187056324874, 5.609986881410345, 0.5676),
        (5.609986881410345, 5.684786706495816, 0.5732),
        (5.684786706495816, 5.759586531581288, 0.5787),
        (5.759586531581288, 5.834386356666759, 0.5843),
        (5.834386356666759, 5.90918618175223, 0.5898),
        (5.90918618175223, 5.983986006837702, 0.5954),
        (5.983986006837702, 6.058785831923172, 0.6010),
        (6.058785831923172, 6.133585657008644, 0.6065),
        (6.133585657008644, 6.208385482094116, 0.6121),
        (6.208385482094116, 6.283185307179586, 0.6176),
    ]

    for start, end, value in coefficients:
        if start <= x <= end:
            return value

def cubic_formula(x):# the same logic with derivative_cubic
    coefficients = [
        [1.0, 0.1532, 0.0, 0.0],                                # 1. aralık (0 <= x <= 0.0748)
        [1.003153810277175, 0.1587, 0.0, 47.624624345020734], # 2. aralık (0.0748 <= x <= 0.1496)
        [1.0126951848095227, 0.1641, 10.686940712306482, -36.95200835048849],  # 3. aralık (0.1496 <= x <= 0.22
        [1.0288681535333022, 0.1696, 2.394929428786245, 32.994628651544225],    # 4. aralık (0.2244 <= x <= 0.2
        [1.0520931463771646, 0.1750, 9.798906784473006, -19.202478652723546],  # 5. aralık (0.2992 <= x <= 0.37
        [1.0829859021764197, 0.1805, 5.489880651179353, 18.685256468071852],  # 6. aralık (0.3740 <= x <= 0.448
        [1.12237754238504, 0.1859, 9.682842397646187, -1.3789267277433406],   # 7. aralık (0.4488 <= x <= 0.523
        [1.171319205485879, 0.1923, 9.373411963523537, 3.4618419817427633],  # 8. aralık (0.5236 <= x <= 0.5984
        [1.2310248203015035, 0.1987, 10.150247487647237, 12.5919764873163],  # 9. aralık (0.5984 <= x <= 0.6732
        [1.3026298979784323, 0.2052, 12.975880403842114, -23.33606162913964],  # 10. aralık (0.6732 <= x <= 0.7
        [1.3864668722100848, 0.2116, 7.73928041971184, -2.8480823123308436],  # 11. aralık (0.7480 <= x <= 0.82
        [1.4802237271514045, 0.2179, 7.100172243337726, -110.5745433351254],  # 12. aralık (0.8228 <= x <= 0.89
        [1.5751599119723292, 0.2234, -17.71269725778199, -91.61333089438563],  # 13. aralık (0.8976 <= x <= 0.9
        [1.6515155710097078, 0.2290, -38.27068063697435, -145.40330272602762],  # 14. aralık (0.9724 <= x <= 1.
        [1.6817928305074292, 0.2345, -70.89910546924446, 151.2028983475595],  # 15. aralık (1.0472 <= x <= 1.12
        [1.6515155710097078, 0.2401, -36.969254422803296, 74.21454402978603],  # 16. aralık (1.1220 <= x <= 1.1
        [1.575159911972329, 0.2457, -20.315549686125276, 139.57252144280835],  # 17. aralık (1.1968 <= x <= 1.2
        [1.4802237271514045, 0.2512, 11.004450885855508, -37.749087038446895],  # 18. aralık (1.2716 <= x <= 1.
        [1.3864668722100852, 0.2568, 2.533575563019351, 75.53242222300155],    # 19. aralık (1.3464 <= x <= 1.4
        [1.3026298979784323, 0.2623, 19.483011474706803, -76.38752832424304],  # 20. aralık (1.4212 <= x <= 1.4
        [1.2310248203015035, 0.2679, 2.3416902026122024, 71.9329010982375],    # 21. aralık (1.4960 <= x <= 1.5
        [1.171319205485879, 0.2734, 18.483395462728176, -85.61500759528502],   # 22. aralık (1.5708 <= x <= 1.6
        [1.12237754238504, 0.2789, -0.728567315727684, 79.9078690980073],      # 23. aralık (1.6456 <= x <= 1.7
        [1.0829859021764199, 0.2845, 17.202716578723376, -90.98983815641958],  # 24. aralık (1.7204 <= x <= 1.7
        [1.0520931463771646, 0.2900, -3.215355357243153, 88.79687940067652],  # 25. aralık (1.7952 <= x <= 1.87
        [1.0288681535333022, 0.2956, 16.309013369128735, -88.27683163600172],  # 26. aralık (1.8700 <= x <= 1.9
        [1.0126951848095227, 0.3011, -2.6884550065693583, 86.2136288975572],  # 27. aralık (1.9448 <= x <= 2.01
        [1.003153810277175, 0.3067, 16.14101364891655, -84.55011043878449],   # 28. aralık (2.0196 <= x <= 2.09
        [1.0, 0.3122, -2.094710056518205, 82.78222416213256]                    # 29. aralık (2.0944 <= x <= 2.
    ]

    result = 0.0

    for i, coeff in enumerate(coefficients):
        result = coeff[0] * x**3 + coeff[1] * x**2 + coeff[2] * x + coeff[3]

    return result


#!!! In the cubic spline method, the array size becomes excessively large for x value 360.
#!!! To eliminate this issue, the interval has been necessarily reduced to 85.

x_interpolation_points = np.linspace(0, 2 * np.pi, 85)
x_values = []
x_values = np.linspace(0, 2 * np.pi, 85)
y_values = []
```

```
for i in range(0, len(x_values)):
    y_values.append(p(x_values[i]))

# Since the intervals have been reduced, new y values for the initial p(x) function have been calculated for th
#for dp/dx:
dp_dx_list_2 = []
for i in range(0, len(x_values)):
    dp_dx_list_2.append(newton_forward(p, x_values[i], h))

#for dy/dx:
def y_function(x):
    y = p(x) * np.sin(x)
    return y

dy_dx_list_2 = []
for i in range(0, len(x_values)):
    dy_dx_list_2.append(newton_forward(y_function, x_values[i], h))

#for d(x)/dx
def x_function(x):
    x = p(x) * np.cos(x)
    return x
dx_dx_list_2 = []

for i in range(0, len(x_values)):
    dx_dx_list_2.append(newton_forward(x_function, x_values[i], h))
#for L(x):

sum_list_2 = []
for i in range(0, len(x_values)):
    sum_list_2.append((dx_dx_list_2[i]**2 + dy_dx_list_2[i]**2)**0.5)


# Perform cubic spline interpolation
y_interp_cubic = cubic_spline_interpolation(x_values, y_values, x_interpolation_points)

# Visualization between p(x) and cubic spline function and comparing...

plt.figure(figsize=(12, 8))
plt.plot(x_values, y_interp_cubic, 'o', label='Data Points', linewidth = 2, color = "black")
plt.plot(x_interpolation_points, y_interp_cubic, label='Cubic Spline Interpolation',  color="blue", linewidth=5
plt.title('Cubic Spline Interpolation')
plt.legend()
plt.plot(x_values, y_values, label = 'Real P(x) Function',  color="red", linewidth=3)
plt.title('real P(x) Funciton')
plt.legend()
plt.tight_layout()
plt.show()

#*****************************************************************************************************
# for the other functions, calculating the functions(dp/dx, dx/dx, dy/dx, l(x)) with cubic_formula and derivati

def x_function_C(x):
    return cubic_formula(x) * np.cos(x)
def y_function_C(x):
    return cubic_formula(x) * np.sin(x)
def derivative_x_C(x):
    return derivative_cubic(x) * np.cos(x) + cubic_formula(x) * -np.sin(x)
def derivative_y_C(x):
    return derivative_cubic(x) * np.sin(x) + cubic_formula(x) * np.cos(x)

def L_function_C(x):
    return ((derivative_x_C(x))**2 + (derivative_y_C(x))**2)**0.5
#for dp/dx
y_val_dp_dx_C = []
for i in range(0, len(x_values)):
    y_val_dp_dx_C.append(derivative_cubic(x_values[i]))


#for dx/dx:
y_val_dx_dx_C = []
for i in range(0, len(x_values)):
    y_val_dx_dx_C.append(derivative_x_C(x))
#for dy/dx:
y_val_dy_dx_C = []
for i in range(0, len(x_values)):
    y_val_dy_dx_C.append(derivative_y_C(x))
#for L function :
y_val_L_C =[]
for i in range(0, len(x_values)):
    y_val_L_C.append(L_function_C(x_values[i]))


# shows the graph has numeric functions and cubic spline functions.....
plt.figure(figsize=(6, 6))
plt.plot(x_values, dp_dx_list_2, label='Numeric Derivative',  color="blue", linewidth=5)
plt.title('Numeric Derivative of p(φ)')
plt.xlabel('φ')
plt.ylabel('dp/dx')
```

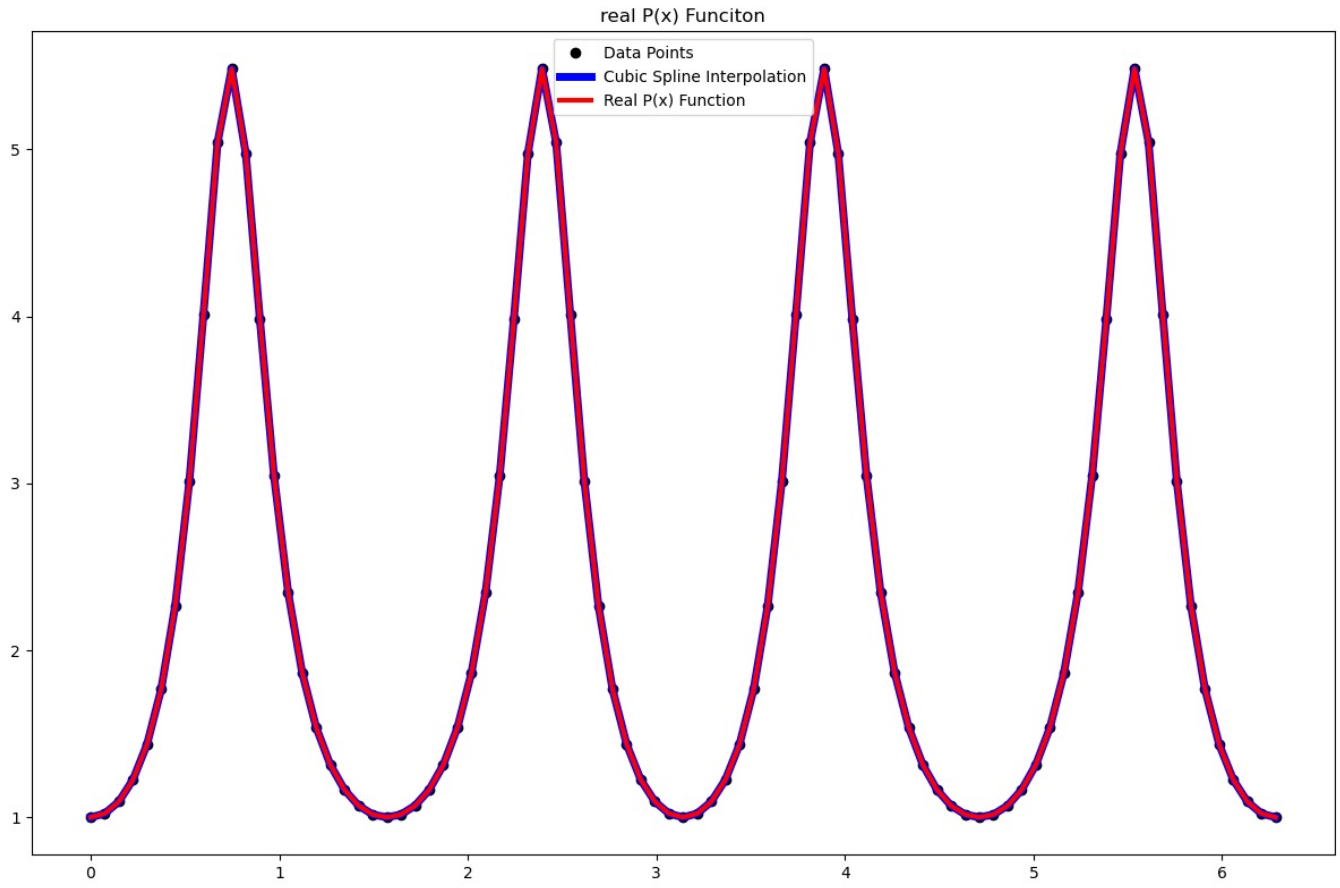
```python
plt.legend()
plt.grid(True)
plt.plot(x_values, y_val_dp_dx_C, label='dp/dx (Cubic Spline)',  color="red", linewidth=3)
plt.title('dp/dx Comparison')
plt.xlabel('φ')
plt.ylabel('dp/dx')
plt.legend()
plt.grid(True)
plt.show()

#****************************************
# finding relative errors and showing graph...
relative_errors_dp_dx_2 = np.abs(np.array(dp_dx_list_2) - np.array(y_val_dp_dx_C))
relative_errors_p_x_2 = np.abs(np.array(np.array(y_values)) - np.array(y_interp_cubic))
plt.subplot(2, 2, 1)
plt.plot(x_values, relative_errors_dp_dx_2, label='Relative Error (dp/dx)')
plt.title('(NUMERİC AND CUBIC SPLINE)')
plt.xlabel('φ')
plt.ylabel('Relative Error')
plt.legend()
plt.grid(True)

plt.subplot(2, 2, 2)
plt.plot(x_values, relative_errors_p_x_2, label='Relative Error p(x)')
plt.title('(NUMERİC AND CUBIC SPLINE)')
plt.xlabel('φ')
plt.ylabel('Relative Error ')
plt.legend()
plt.grid(True)
plt.show()
```

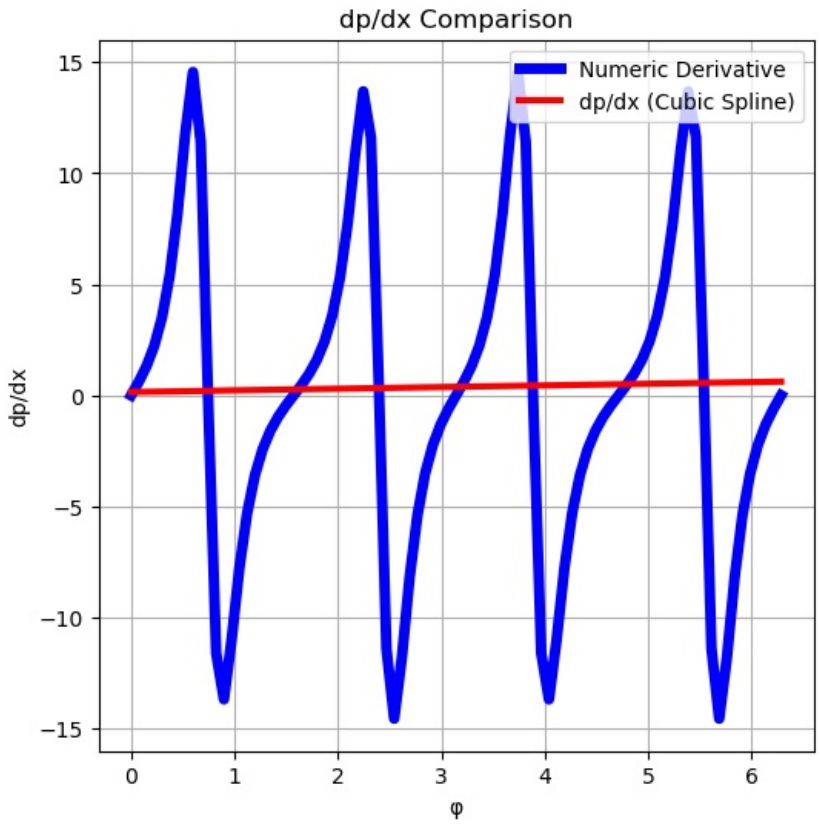
```
S'0(x) = 0.3028557784588177  for  0.0 <= x <= 0.07479982508547127
S'1(x) = 0.3028557784588177  for  0.07479982508547127 <= x <= 0.14959965017094254
S'2(x) = -1.7343013893910024  for  0.14959965017094254 <= x <= 0.2243994752564138
S'3(x) = 16.45458141992645  for  0.2243994752564138 <= x <= 0.2991993003418851
S'4(x) = -4.0749550767787  for  0.2991993003418851 <= x <= 0.37399912542735636
S'5(x) = 31.686492661278407  for  0.37399912542735636 <= x <= 0.4487989505128276
S'6(x) = 0.4225076244654531  for  0.4487989505128276 <= x <= 0.5235987755982989
S'7(x) = 53.83208357016086  for  0.5235987755982989 <= x <= 0.5983986006837702
S'8(x) = -0.3327011799575246  for  0.5983986006837702 <= x <= 0.6731984257692414
S'9(x) = 25.03208234551694  for  0.6731984257692414 <= x <= 0.7479982508547127
S'10(x) = -105.5533368110467  for  0.7479982508547127 <= x <= 0.822798075940184
S'11(x) = -17.001715012641228  for  0.822798075940184 <= x <= 0.8975979010256552
S'12(x) = -53.38337665694516  for  0.8975979010256552 <= x <= 0.9723977261111265
S'13(x) = 45.11443867437248  for  0.9723977261111265 <= x <= 1.0471975511965979
S'14(x) = -36.67844636744621  for  1.0471975511965979 <= x <= 1.121997376282069
S'15(x) = 50.06329699222462  for  1.121997376282069 <= x <= 1.1967972013675403
S'16(x) = -43.36954581180066  for  1.1967972013675403 <= x <= 1.2715970264530116
S'17(x) = 51.215685135469386  for  1.2715970264530116 <= x <= 1.3463968515384828
S'18(x) = -48.68905298438668  for  1.3463968515384828 <= x <= 1.421196676623954
S'19(x) = 54.19536510419505  for  1.421196676623954 <= x <= 1.4959965017094254
S'20(x) = -52.5907527080015  for  1.4959965017094254 <= x <= 1.5707963267948966
S'21(x) = 58.244338876687095  for  1.5707963267948966 <= x <= 1.645596151880368
S'22(x) = -55.571710852475064  for  1.645596151880368 <= x <= 1.7203959769658392
S'23(x) = 63.26305220362639  for  1.7203959769658392 <= x <= 1.7951958020513104
S'24(x) = -57.32712810178788  for  1.7951958020513104 <= x <= 1.8699956271367817
S'25(x) = 70.02579480184698  for  1.8699956271367817 <= x <= 1.944795452222253
S'26(x) = -56.447509119385245  for  1.944795452222253 <= x <= 2.019595277307724
S'27(x) = 80.80745280480312  for  2.019595277307724 <= x <= 2.0943951023931957
S'28(x) = -49.837845661860115  for  2.0943951023931957 <= x <= 2.169194927478667
S'29(x) = 97.56928579130607  for  2.169194927478667 <= x <= 2.243994752564138
S'30(x) = -46.22634960210462  for  2.243994752564138 <= x <= 2.3187945776496095
S'31(x) = 74.53594340119884  for  2.3187945776496095 <= x <= 2.3935944027350806
S'32(x) = -138.82682877773004  for  2.3935944027350806 <= x <= 2.4683942278205517
S'33(x) = 11.05692287580871  for  2.4683942278205517 <= x <= 2.5431940529060233
S'34(x) = -91.63804833431675  for  2.5431940529060233 <= x <= 2.6179938779914944
S'35(x) = 77.62424862458346  for  2.6179938779914944 <= x <= 2.6927937030769655
S'36(x) = -65.57059606789585  for  2.6927937030769655 <= x <= 2.767593528162437
S'37(x) = 79.77463583670215  for  2.767593528162437 <= x <= 2.842393353247908
S'38(x) = -69.26338793289375  for  2.842393353247908 <= x <= 2.9171931783333793
S'39(x) = 76.7914088211266  for  2.9171931783333793 <= x <= 2.991993003418851
S'40(x) = -71.20183115198046  for  2.991993003418851 <= x <= 3.066792828504322
S'41(x) = 76.22623187870124  for  3.066792828504322 <= x <= 3.141592653589793
S'42(x) = -71.17454620275305  for  3.141592653589793 <= x <= 3.2163924786752647
S'43(x) = 77.3156795835755  for  3.2163924786752647 <= x <= 3.291192303760736
S'44(x) = -69.44761200808722  for  3.291192303760736 <= x <= 3.365992128846207
S'45(x) = 80.56426038002547  for  3.365992128846207 <= x <= 3.4407919539316785
S'46(x) = -64.58100237828211  for  3.4407919539316785 <= x <= 3.5155917790171496
S'47(x) = 88.5889083041842  for  3.5155917790171496 <= x <= 3.5903916041026207
S'48(x) = -52.8762763598433  for  3.5903916041026207 <= x <= 3.6651914291880923
S'49(x) = 103.52723589587328  for  3.6651914291880923 <= x <= 3.7399912542735634
S'50(x) = -46.42422184707141  for  3.7399912542735634 <= x <= 3.814791079359035
S'51(x) = 67.51997135403394  for  3.814791079359035 <= x <= 3.889590904444506
S'52(x) = -144.4375941609647  for  3.889590904444506 <= x <= 3.964390729529977
S'53(x) = 18.278910678678244  for  3.964390729529977 <= x <= 4.039190554615448
S'54(x) = -85.06037068966617  for  4.039190554615448 <= x <= 4.11399037970092
S'55(x) = 73.18780104849594  for  4.11399037970092 <= x <= 4.188790204786391
S'56(x) = -61.148177082971756  for  4.188790204786391 <= x <= 4.263590029871862
S'57(x) = 70.92939604914983  for  4.263590029871862 <= x <= 4.338389854957334
```

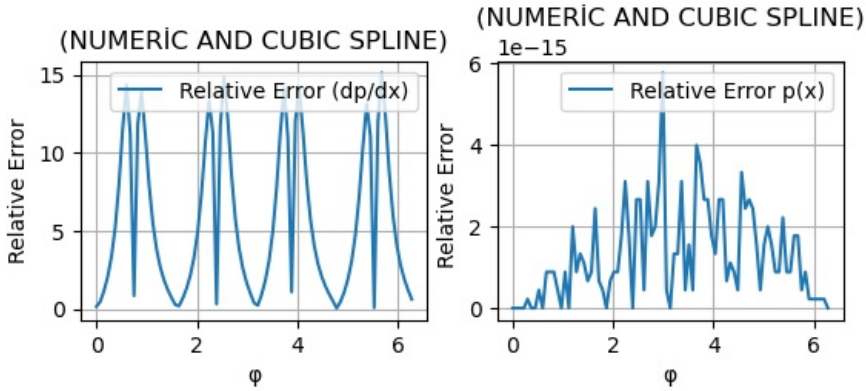
S'58(x) = -60.63201321012917 for 4.338389854957334 <= x <= 4.413189680042805
S'59(x) = 64.87452087519972 for 4.413189680042805 <= x <= 4.487989505128276
S'60(x) = -58.74425706551682 for 4.487989505128276 <= x <= 4.562789330213747
S'61(x) = 60.646937526728266 for 4.562789330213747 <= x <= 4.637589155299219
S'62(x) = -55.43869347193652 for 4.637589155299219 <= x <= 4.71238898038469
S'63(x) = 57.48864798202226 for 4.71238898038469 <= x <= 4.787188805470161
S'64(x) = -51.21238829921299 for 4.787188805470161 <= x <= 4.861988630555633
S'65(x) = 55.300097991766165 for 4.861988630555633 <= x <= 4.9367884556411035
S'66(x) = -45.76054223132786 for 4.9367884556411035 <= x <= 5.011588280726575
S'67(x) = 54.85557727278959 for 5.011588280726575 <= x <= 5.086388105812047
S'68(x) = -37.673659931729844 for 5.086388105812047 <= x <= 5.161187930897517
S'69(x) = 58.42997195854852 for 5.161187930897517 <= x <= 5.235987755982989
S'70(x) = -23.856733157007056 for 5.235987755982989 <= x <= 5.31078758106846
S'71(x) = 67.98454162785481 for 5.31078758106846 <= x <= 5.385587406153931
S'72(x) = -13.037973780051942 for 5.385587406153931 <= x <= 5.460387231239403
S'73(x) = 37.743935920545 for 5.460387231239403 <= x <= 5.535187056324874
S'74(x) = -98.43118963847624 for 5.535187056324874 <= x <= 5.609986881410345
S'75(x) = -32.942347922047645 for 5.609986881410345 <= x <= 5.684786706495816
S'76(x) = -44.03514587785807 for 5.684786706495816 <= x <= 5.759586531581288
S'77(x) = 26.41771450952399 for 5.759586531581288 <= x <= 5.834386356666759
S'78(x) = -10.760430294234794 for 5.834386356666759 <= x <= 5.90918618175223
S'79(x) = 21.360838404439733 for 5.90918618175223 <= x <= 5.983986006837702
S'80(x) = -7.245958842031641 for 5.983986006837702 <= x <= 6.058785831923172
S'81(x) = 11.170348071663355 for 6.058785831923172 <= x <= 6.133585657008644
S'82(x) = -1.977138743915333 for 6.133585657008644 <= x <= 6.208385482094116
S'83(x) = 3.3979078120364505 for 6.208385482094116 <= x <= 6.283185307179586
S0(x) = 1.0 + 0.3028557784588177 * (x - 0.0) + 0.0 * (x - 0.0)^2 + 0.0 * (x - 0.0)^3 for 0.0 <= x <= 0.07479982508547127
S1(x) = 1.0226535592548438 + 2.2979737157860445 * (x - 0.07479982508547127) + 0.0 * (x - 0.07479982508547127)^2 + -240.2302732197054 * (x - 0.07479982508547127)^3 for 0.07479982508547127 <= x <= 0.14959965017094254
S2(x) = 1.094003767058736 + -3.5831265570558717 * (x - 0.14959965017094254) + -53.90754725120681 * (x - 0.14959965017094254)^2 + 1674.2441887873401 * (x - 0.14959965017094254)^3 for 0.14959965017094254 <= x <= 0.2243994752564138
S3(x) = 1.225054387074612 + -5.745698997543647 * (x - 0.2243994752564138) + 321.7919701637725 * (x - 0.2243994752564138)^2 + -2768.4901611855635 * (x - 0.2243994752564138)^3 for 0.2243994752564138 <= x <= 0.2991993003418851
S4(x) = 1.4370777328617819 + 1.9569259947627686 * (x - 0.2991993003418851) + -299.45576925881255 * (x - 0.2991993003418851)^2 + 4440.146602925048 * (x - 0.2991993003418851)^3 for 0.2991993003418851 <= x <= 0.37399912542735636
S5(x) = 1.7662245267093437 + -16.20742178402215 * (x - 0.37399912542735636) + 696.9107984991167 * (x - 0.37399912542735636)^2 + -5220.5818052590275 * (x - 0.37399912542735636)^3 for 0.37399912542735636 <= x <= 0.4487989505128276
S6(x) = 2.268292668438287 + 5.742629900668863 * (x - 0.4487989505128276) + -474.5850191341898 * (x - 0.4487989505128276)^2 + 7094.841351409501 * (x - 0.4487989505128276)^3 for 0.4487989505128276 <= x <= 0.5235987755982989
S7(x) = 3.0117647058823542 + -21.53553034280813 * (x - 0.5235987755982989) + 1117.4936571496094 * (x - 0.5235987755982989)^2 + -8696.661907475584 * (x - 0.5235987755982989)^3 for 0.5235987755982989 <= x <= 0.5983986006837702
S8(x) = 4.013697597296041 + 39.28857793799302 * (x - 0.5983986006837702) + -834.032711370353 * (x - 0.5983986006837702)^2 + 6584.106823319344 * (x - 0.5983986006837702)^3 for 0.5983986006837702 <= x <= 0.6731984257692414
S9(x) = 5.041540376985465 + 37.553299635061165 * (x - 0.6731984257692414) + 643.4374048146806 * (x - 0.6731984257692414)^2 + -14260.593108177207 * (x - 0.6731984257692414)^3 for 0.6731984257692414 <= x <= 0.7479982508547127
S10(x) = 5.482417222987358 + 93.95585415231328 * (x - 0.7479982508547127) + -2556.632205505519 * (x - 0.7479982508547127)^2 + 16175.933459532302 * (x - 0.7479982508547127)^3 for 0.7479982508547127 <= x <= 0.822798075940184
S11(x) = 4.975632239110864 + -33.40421666123434 * (x - 0.822798075940184) + 1073.2387745961942 * (x - 0.822798075940184)^2 + -10755.72587901258 * (x - 0.822798075940184)^3 for 0.822798075940184 <= x <= 0.8975979010256552
S12(x) = 3.980452047982537 + 8.833870747593913 * (x - 0.8975979010256552) + -1340.3404686560575 * (x - 0.8975979010256552)^2 + 14107.503587020032 * (x - 0.8975979010256552)^3 for 0.8975979010256552 <= x <= 0.9723977261111265
S13(x) = 3.0460762587666874 + -63.91605482224752 * (x - 0.9723977261111265) + 1825.375933449216 * (x - 0.9723977261111265)^2 + -14646.25468113032 * (x - 0.9723977261111265)^3 for 0.9723977261111265 <= x <= 1.0471975511965979
S14(x) = 2.3486238532110084 + 19.92386257825248 * (x - 1.0471975511965979) + -1461.2359314682255 * (x - 1.0471975511965979)^2 + 14819.136348383836 * (x - 1.0471975511965979)^3 for 1.0471975511965979 <= x <= 1.121997376282069
S15(x) = 1.8651883466799504 + -54.63271760664754 * (x - 1.121997376282069) + 1864.1704888623485 * (x - 1.121997376282069)^2 + -15943.724727916067 * (x - 1.121997376282069)^3 for 1.121997376282069 <= x <= 1.1967972013675403
S16(x) = 1.5361849431175731 + 34.01202180668017 * (x - 1.1967972013675403) + -1713.5929737147292 * (x - 1.1967972013675403)^2 + 16297.64053649423 * (x - 1.1967972013675403)^3 for 1.1967972013675403 <= x <= 1.2715970264530116
S17(x) = 1.313363097084041 + -51.32955537459272 * (x - 1.2715970264530116) + 1943.5890105922379 * (x - 1.2715970264530116)^2 + -17165.26787837049 * (x - 1.2715970264530116)^3 for 1.2715970264530116 <= x <= 1.3463968515384828
S18(x) = 1.1645601721828371 + 42.370136559984644 * (x - 1.3463968515384828) + -1908.2880939498675 * (x - 1.3463968515384828)^2 + 17712.464985678856 * (x - 1.3463968515384828)^3 for 1.3463968515384828 <= x <= 1.421196676623954
S19(x) = 1.0697145570682116 + -52.045308547273535 * (x - 1.421196676623954) + 2066.379754334075 * (x - 1.421196676623954)^2 + -18449.470771817654 * (x - 1.421196676623954)^3 for 1.421196676623954 <= x <= 1.4959965017094254
S20(x) = 1.016942641724772 + 48.09321508157316 * (x - 1.4959965017094254) + -2073.6718056203545 * (x - 1.4959965017094254)^2 + 19086.738660421313 * (x - 1.4959965017094254)^3 for 1.4959965017094254 <= x <= 1.5707963267948966
S21(x) = 1.0 + -54.50509099123637 * (x - 1.5707963267948966) + 2209.382334134486 * (x - 1.5707963267948966)^2 + -19755.05655867534 * (x - 1.5707963267948966)^3 for 1.5707963267948966 <= x <= 1.645596151880368
S22(x) = 1.016942641724772 + 52.59075270800136 * (x - 1.645596151880368) + -2223.64199129304 * (x - 1.645596151880368)^2 + 20454.42446657993 * (x - 1.645596151880368)^3 for 1.645596151880368 <= x <= 1.7203959769658392

S23(x) = 1.0697145570682116 + -57.934612989646006 * (x - 1.7203959769658392) + 2366.3201256794528 * (x - 1.7203959769658392)^2 + -21054.05447694938 * (x - 1.7203959769658392)^3 for 1.7203959769658392 <= x <= 1.7951958020513104
S24(x) = 1.1645601721828371 + 56.16754875528864 * (x - 1.7951958020513104) + -2358.198650967928 * (x - 1.7951958020513104)^2 + 21843.493166149034 * (x - 1.7951958020513104)^3 for 1.7951958020513104 <= x <= 1.8699956271367817
S25(x) = 1.313363097084041 + -62.43342879182196 * (x - 1.8699956271367817) + 2543.4697532829805 * (x - 1.8699956271367817)^2 + -22312.50162078098 * (x - 1.8699956271367817)^3 for 1.8699956271367817 <= x <= 1.944795452222253
S26(x) = 1.5361849431175731 + 58.32653735360403 * (x - 1.944795452222253) + -2463.4439020781583 * (x - 1.944795452222253)^2 + 23295.221608711043 * (x - 1.944795452222253)^3 for 1.944795452222253 <= x <= 2.019595277307724
S27(x) = 1.8651883466799504 + -68.75953641947882 * (x - 2.019595277307724) + 2763.9916028984608 * (x - 2.019595277307724)^2 + -23507.26902568685 * (x - 2.019595277307724)^3 for 2.019595277307724 <= x <= 2.0943951023931957
S28(x) = 2.34862385321101 + 59.11393368015116 * (x - 2.0943951023931957) + -2511.027231177039 * (x - 2.0943951023931957)^2 + 24671.02315494183 * (x - 2.0943951023931957)^3 for 2.0943951023931957 <= x <= 2.169194927478667
S29(x) = 3.04607625876669 + -71.28917387252817 * (x - 2.169194927478667) + 3025.1374188307313 * (x - 2.169194927478667)^2 + -25468.90785733992 * (x - 2.169194927478667)^3 for 2.169194927478667 <= x <= 2.243994752564138
S30(x) = 3.9804520479825345 + 83.2973597405521 * (x - 2.243994752564138) + -2690.07213971029 * (x - 2.243994752564138)^2 + 23453.765945840918 * (x - 2.243994752564138)^3 for 2.243994752564138 <= x <= 2.3187945776496095
S31(x) = 4.975632239110869 + -16.65151595641779 * (x - 2.3187945776496095) + 2572.9406313231802 * (x - 2.3187945776496095)^2 + -30210.609322869583 * (x - 2.3187945776496095)^3 for 2.3187945776496095 <= x <= 2.3935944027350806
S32(x) = 5.482417222987358 + 142.94581566555792 * (x - 2.3935944027350806) + -4206.304247905263 * (x - 2.3935944027350806)^2 + 29631.904768023338 * (x - 2.3935944027350806)^3 for 2.3935944027350806 <= x <= 2.4683942278205517
S33(x) = 5.041540376985467 + -66.16380908548108 * (x - 2.4683942278205517) + 2443.0796328871866 * (x - 2.4683942278205517)^2 + -23292.054279674227 * (x - 2.4683942278205517)^3 for 2.4683942278205517 <= x <= 2.5431940529060233
S34(x) = 4.013697597296038 + 45.20367580537503 * (x - 2.5431940529060233) + -2783.6451251156377 * (x - 2.5431940529060233)^2 + 26741.245160339506 * (x - 2.5431940529060233)^3 for 2.5431940529060233 <= x <= 2.6179938779914944
S35(x) = 3.0117647058823516 + -102.44230608103126 * (x - 2.6179938779914944) + 3217.07625656765 * (x - 2.6179938779914944)^2 + -26476.06040078215 * (x - 2.6179938779914944)^3 for 2.6179938779914944 <= x <= 2.6927937030769655
S36(x) = 2.268292668438289 + 51.926962771857504 * (x - 2.6927937030769655) + -2724.13780422497 * (x - 2.6927937030769655)^2 + 25938.43665114018 * (x - 2.6927937030769655)^3 for 2.6927937030769655 <= x <= 2.76759352816243 7
S37(x) = 1.7662245267093422 + -87.7752109430537 * (x - 2.767593528162437) + 3096.4337692626405 * (x - 2.767593528162437)^2 + -26494.637245314967 * (x - 2.767593528162437)^3 for 2.767593528162437 <= x <= 2.842393353247908
S38(x) = 1.4370777328617808 + 63.90292124400575 * (x - 2.842393353247908) + -2848.9489256950624 * (x - 2.842393353247908)^2 + 26159.616600084028 * (x - 2.842393353247908)^3 for 2.842393353247908 <= x <= 2.9171931783333793
S39(x) = 1.225054387074612 + -80.02179547260548 * (x - 2.9171931783333793) + 3021.2553122727536 * (x - 2.9171931783333793)^2 + -26402.006424194307 * (x - 2.9171931783333793)^3 for 2.9171931783333793 <= x <= 2.99199300341 8851
S40(x) = 1.094003767058736 + 69.04076332752254 * (x - 2.991993003418851) + -2903.3410750329376 * (x - 2.991993003418851)^2 + 26304.62830513547 * (x - 2.991993003418851)^3 for 2.991993003418851 <= x <= 3.066792828504322
S41(x) = 1.0226535592548438 + -77.04444712986476 * (x - 3.066792828504322) + 2999.4037134544596 * (x - 3.066792828504322)^2 + -26383.005688008692 * (x - 3.066792828504322)^3 for 3.066792828504322 <= x <= 3.14159265358979 3
S42(x) = 1.0 + 71.03892997589834 * (x - 3.141592653589793) + -2920.9289186216592 * (x - 3.141592653589793)^2 + 26407.244453256924 * (x - 3.141592653589793)^3 for 3.141592653589793 <= x <= 3.2163924786752647
S43(x) = 1.0226535592548438 + -76.22623187870164 * (x - 3.2163924786752647) + 3004.8428796570615 * (x - 3.2163924786752647)^2 + -26377.344600880566 * (x - 3.2163924786752647)^3 for 3.2163924786752647 <= x <= 3.291192303760736
S44(x) = 1.094003767058736 + 71.33744737883461 * (x - 3.291192303760736) + -2914.2194074381255 * (x - 3.291192303760736)^2 + 26523.20025043629 * (x - 3.291192303760736)^3 for 3.291192303760736 <= x <= 3.365992128846207
S45(x) = 1.225054387074612 + -77.0626412748369 * (x - 3.365992128846207) + 3037.5728108805497 * (x - 3.365992128846207)^2 + -26329.287956822554 * (x - 3.365992128846207)^3 for 3.365992128846207 <= x <= 3.4407919539316785
S46(x) = 1.4370777328617819 + 69.67023661345931 * (x - 3.4407919539316785) + -2870.70559050547 * (x - 3.4407919539316785)^2 + 26712.786132550427 * (x - 3.4407919539316785)^3 for 3.4407919539316785 <= x <= 3.5155917790171496
S47(x) = 1.7662245267093437 + -80.31710074412179 * (x - 3.5155917790171496) + 3123.629600275641 * (x - 3.5155917790171496)^2 + -26205.063068872565 * (x - 3.5155917790171496)^3 for 3.5155917790171496 <= x <= 3.5903916041026207
S48(x) = 2.268292668438287 + 66.24867720217172 * (x - 3.5903916041026207) + -2756.772801440579 * (x - 3.5903916041026207)^2 + 26791.16434901107 * (x - 3.5903916041026207)^3 for 3.5903916041026207 <= x <= 3.66519142918809 23
S49(x) = 3.0117647058823542 + -78.43794598571445 * (x - 3.6651914291880923) + 3255.150419985869 * (x - 3.6651914291880923)^2 + -27104.82663906538 * (x - 3.6651914291880923)^3 for 3.6651914291880923 <= x <= 3.739991254273 5634
S50(x) = 4.013697597296041 + 92.58736192230211 * (x - 3.7399912542735634) + -2827.1584547364564 * (x - 3.7399912542735634)^2 + 23704.113288897 * (x - 3.7399912542735634)^3 for 3.7399912542735634 <= x <= 3.814791079359035
S51(x) = 5.041540376985469 + -12.14185269065095 * (x - 3.814791079359035) + 2492.032128710636 * (x - 3.814791079359035)^2 + -30092.44130774296 * (x - 3.814791079359035)^3 for 3.814791079359035 <= x <= 3.889590904444506
S52(x) = 5.482417222987359 + 140.04737481942755 * (x - 3.889590904444506) + -4260.695909931301 * (x - 3.889590904444506)^2 + 30719.623393085778 * (x - 3.889590904444506)^3 for 3.889590904444506 <= x <= 3.964390729529977
S53(x) = 4.975632239110864 + -75.89210566975055 * (x - 3.964390729529977) + 2632.771459551788 * (x - 3.964390729529977)^2 + -24011.257546553712 * (x - 3.964390729529977)^3 for 3.964390729529977 <= x <= 4.039190554615448
S54(x) = 3.98045204798254 + 47.718128097511794 * (x - 4.039190554615448) + -2755.342134141459 * (x - 4.039190554615448)^2 + 26074.876988548724 * (x - 4.039190554615448)^3 for 4.039190554615448 <= x <= 4.11399037970092
S55(x) = 3.0460762587666874 + -99.19668051356723 * (x - 4.11399037970092) + 3095.846579464436 * (x - 4.11399037970092)^2 + -25325.469816646775 * (x - 4.11399037970092)^3 for 4.11399037970092 <= x <= 4.188790204786391
S56(x) = 2.3486238532110066 + 51.60085661097415 5 * (x - 4.188790204786391) + -2587.175558013268 * (x - 4.188790204786391)^2 + 24210.19321788818 * (x - 4.188790204786391)^3 for 4.188790204786391 <= x <= 4.263590029871862
S57(x) = 1.8651883466799515 + -82.70607998077053 * (x - 4.263590029871862) + 2845.5790959371852 * (x - 4.263590029871862)^2 + -24046.623331407805 * (x - 4.263590029871862)^3 for 4.263590029871862 <= x <= 4.33838985495733 4

S58(x) = 1.5361849431175731 + 58.48175252220504 * (x - 4.338389854957334) + -2550.4705613193846 * (x - 4.338389854957334)^2 + 23112.380873973703 * (x - 4.338389854957334)^3 for 4.338389854957334 <= x <= 4.413189680042805
S59(x) = 1.3133630970840402 + -72.19565443151936 * (x - 4.413189680042805) + 2635.935578726709 * (x - 4.413189680042805)^2 + -22691.849949837782 * (x - 4.413189680042805)^3 for 4.413189680042805 <= x <= 4.487989505128276
S60(x) = 1.1645601721828371 + 59.63260395831246 * (x - 4.487989505128276) + -2456.103642614129 * (x - 4.487989505128276)^2 + 21950.888791133544 * (x - 4.487989505128276)^3 for 4.487989505128276 <= x <= 4.562789330213747
S61(x) = 1.0697145570682116 + -65.7041442870031 * (x - 4.562789330213747) + 2469.6642835281527 * (x - 4.562789330213747)^2 + -21399.73631126007 * (x - 4.562789330213747)^3 for 4.562789330213747 <= x <= 4.637589155299219
S62(x) = 1.016942641724772 + 58.14841916270458 * (x - 4.637589155299219) + -2332.4253153442496 * (x - 4.637589155299219)^2 + 20748.84593385155 * (x - 4.637589155299219)^3 for 4.637589155299219 <= x <= 4.71238898038469
S63(x) = 1.0 + -60.95666341376913 * (x - 4.71238898038469) + 2323.604824388176 * (x - 4.71238898038469)^2 + -20129.005566093005 * (x - 4.71238898038469)^3 for 4.71238898038469 <= x <= 4.787188805470161
S64(x) = 1.016942641724772 + 55.43869347193586 * (x - 4.787188805470161) + -2193.3334620765445 * (x - 4.787188805470161)^2 + 19540.2152079853 * (x - 4.787188805470161)^3 for 4.787188805470161 <= x <= 4.861988630555633
S65(x) = 1.0697145570682116 + -57.17892209498224 * (x - 4.861988630555633) + 2191.4805769927707 * (x - 4.861988630555633)^2 + -18851.686952342516 * (x - 4.861988630555633)^3 for 4.861988630555633 <= x <= 4.936788455641103 5
S66(x) = 1.1645601721828371 + 51.80822620202614 * (x - 4.9367884556411035) + -2038.8280828110392 * (x - 4.9367884556411035)^2 + 18352.96737552961 * (x - 4.9367884556411035)^3 for 4.9367884556411035 <= x <= 5.011588280726575
S67(x) = 1.3133630970840422 + -54.47047457996099 * (x - 5.011588280726575) + 2079.568165655903 * (x - 5.011588280726575)^2 + -17533.81756414927 * (x - 5.011588280726575)^3 for 5.011588280726575 <= x <= 5.086388105812047
S68(x) = 1.536184943117574 + 46.75995148314488 * (x - 5.086388105812047) + -1855.0112949809 * (x - 5.086388105812047)^2 + 17228.379286067153 * (x - 5.086388105812047)^3 for 5.086388105812047 <= x <= 5.161187930897517
S69(x) = 1.8651883466799488 + -53.58931889042144 * (x - 5.161187930897517) + 2011.0279763310073 * (x - 5.161187930897517)^2 + -16152.2684370305 * (x - 5.161187930897517)^3 for 5.161187930897517 <= x <= 5.235987755982989
S70(x) = 2.34862385321101 + 40.34008449249551 * (x - 5.235987755982989) + -1613.5325851393866 * (x - 5.235987755982989)^2 + 16027.86430027292 * (x - 5.235987755982989)^3 for 5.235987755982989 <= x <= 5.31078758106846
S71(x) = 3.046076258766692 + -48.911693026274435 * (x - 5.31078758106846) + 1983.1117533228787 * (x - 5.31078758106846)^2 + -15537.59073665857 * (x - 5.31078758106846)^3 for 5.31078758106846 <= x <= 5.385587406153931
S72(x) = 3.9804520479825354 + 57.31624723569962 * (x - 5.385587406153931) + -1503.5154547321927 * (x - 5.385587406153931)^2 + 12234.290559146584 * (x - 5.385587406153931)^3 for 5.385587406153931 <= x <= 5.460387231239403
S73(x) = 4.975632239110865 + 12.933228207034746 * (x - 5.460387231239403) + 1241.852926874809 * (x - 5.460387231239403)^2 + -17702.97567016195 * (x - 5.460387231239403)^3 for 5.460387231239403 <= x <= 5.535187056324874
S74(x) = 5.482417222987358 + 109.7574398435047 * (x - 5.535187056324874) + -2730.685523986609 * (x - 5.535187056324874)^2 + 15836.112849302624 * (x - 5.535187056324874)^3 for 5.535187056324874 <= x <= 5.609986881410345
S75(x) = 5.041540376985466 + -29.37180160482729 * (x - 5.609986881410345) + 822.9298894982235 * (x - 5.609986881410345)^2 + -8208.104094940058 * (x - 5.609986881410345)^3 for 5.609986881410345 <= x <= 5.684786706495816
S76(x) = 4.013697597296041 + 4.808036666119884 * (x - 5.684786706495816) + -1018.964362256354 * (x - 5.684786706495816)^2 + 10369.136709591708 * (x - 5.684786706495816)^3 for 5.684786706495816 <= x <= 5.759586531581288
S77(x) = 3.011764705882349 + -58.44303528317429 * (x - 5.759586531581288) + 1307.8644742380513 * (x - 5.759586531581288)^2 + -8815.793684021013 * (x - 5.759586531581288)^3 for 5.759586531581288 <= x <= 5.834386356666759
S78(x) = 2.268292668438289 + 4.3240603153987776 * (x - 5.834386356666759) + -670.3950024250556 * (x - 5.834386356666759)^2 + 6990.011668365695 * (x - 5.834386356666759)^3 for 5.834386356666759 <= x <= 5.90918618175223
S79(x) = 1.7662245267093437 + -36.56867682799363 * (x - 5.90918618175223) + 898.1599479924221 * (x - 5.90918618175223)^2 + -6258.053996527082 * (x - 5.90918618175223)^3 for 5.90918618175223 <= x <= 5.983986006837702
S80(x) = 1.4370777328617808 + 9.092755470345121 * (x - 5.983986006837702) + -506.1440849545637 * (x - 5.983986006837702)^2 + 4634.875085283018 * (x - 5.983986006837702)^3 for 5.983986006837702 <= x <= 6.058785831923172
S81(x) = 1.225054387074613 + -21.60799804034401 * (x - 6.058785831923172) + 533.9194520619634 * (x - 6.058785831923172)^2 + -3589.106643380101 * (x - 6.058785831923172)^3 for 6.058785831923172 <= x <= 6.133585657008644
S82(x) = 1.094003767058736 + 7.023334236659739 * (x - 6.133585657008644) + -271.47419535184315 * (x - 6.133585657008644)^2 + 2203.5702583078582 * (x - 6.133585657008644)^3 for 6.133585657008644 <= x <= 6.208385482094116
S83(x) = 1.0226535592548438 + -11.423386380401164 * (x - 6.208385482094116) + 223.0058143030824 * (x - 6.208385482094116)^2 + -993.7893751679359 * (x - 6.208385482094116)^3 for 6.208385482094116 <= x <= 6.283185307179586


real P(x) Funciton
Data Points
Cubic Spline Interpolation
Real P(x) Function
1
2
3
4
5
0
1
2
3
4
5
6





As seen, the solution has been found with an average error of approximately 2. It is challenging to decide whether the Hermite function or the cubic spline method is more reasonable (because cubic spline requires a more powerful computer to match the interval with that of Hermite). However, for an engineer responsible for making the most efficient use of available resources, choosing the Hermite function is appropriate.

G) In Section G, a function is provided, and it is requested to calculate the integral of this function over the interval [0, π/2] using the Trapezoidal, Simpson, and Gauss quadrature methods separately. Various research has been conducted, and references have been added to the end of the report. Based on the findings of this research, algorithms have been developed using ChatGPT alongside discussions on the logic of these algorithms, primarily on YouTube. This collaborative approach was taken because the number of series and calculations becomes cumbersome at some point. The references for these investigations have also been included at the end of the report. First, the given functions are written in code. Later, the integrals for each of the three methods are calculated with the described functions, and the results are printed with explanations

In [17]:
```
#creating f(x) function according to the report...
def l_function(x, x_function, y_function, h):
```

```python
    return (newton_forward(x_function, x, h)**2 + newton_forward(y_function, x, h)**2)**0.5

def f(x, p, x_function, y_function, h):
    return np.log(p(x)) * l_function(x, x_function, y_function, h)

# obtaining the gauss quadrature function....
def i_gauss(func, a, b, n, p, x_function, y_function, h):
    nod, mean = np.polynomial.legendre.leggauss(n)
    total = 0.0

    for i in range(n):
        x = 0.5 * (b - a) * nod[i] + 0.5 * (a + b)
        total += mean[i] * func(x, p, x_function, y_function, h)

    return 0.5 * (b - a) * total

# obtaining the simpsons function....
def i_simpson(func, a, b, n, p, x_function, y_function, h):
    h = (b - a) / n
    total = func(a, p, x_function, y_function, h) + func(b, p, x_function, y_function, h)

    for i in range(1, n, 2):
        total += 4 * func(a + i * h, p, x_function, y_function, h)

    for i in range(2, n-1, 2):
        total += 2 * func(a + i * h, p, x_function, y_function, h)

    return (h / 3) * total

# obtaining trapezoidal function....
def i_trapezoidal(func, a, b, n, p, x_function, y_function, h):
    h = (b - a) / n
    total = 0.5 * (func(a, p, x_function, y_function, h) + func(b, p, x_function, y_function, h))

    for i in range(1, n):
        total += func(a + i * h, p, x_function, y_function, h)

    return h * total


# limit of total....
(a, b) = (0, np.pi/2)

# creating subregions...
n = 1000


rval_trap = i_trapezoidal(f, a, b, n, p, x_function, y_function, h)
rval_simp = i_simpson(f, a, b, n, p, x_function, y_function, h)
rval_gauss = i_gauss(f, a, b, n, p, x_function, y_function, h)

# showing the result....
print("The Integral , according to Trapezoidal Rule: {}".format(rval_trap))
print("The Integral , according to Simpson Rule :   {}".format(rval_simp))
print("The Integral , according to Gauss Quad. Rule: {}".format(rval_gauss))
```

```
The Integral , according to Trapezoidal Rule: 10.670436693861594
The Integral , according to Simpson Rule :   10.670436693861525
The Integral , according to Gauss Quad. Rule: 10.671311412696344
```

As observed, the algorithms have been correctly implemented (without using libraries), and the integral has been calculated separately for each of the three methods. The result is that as the value of n increases, the calculated values approach the actual values

H) Finally, In this project, it has been determined that issues and challenges arise when drawing functions, the extent towhich data can approach accuracy, and the significance of the processor used are key factors. Regarding the internal structure of the given task, the graphing of the given function and its directional trends in space were initially visualized. Subsequently, the methods of using forward and backward difference techniques to derive derivative functions from this original function were learned. Later on, attempting to rediscover the desired values using Hermite and cubic spline methods contributed to a better understanding of the logic behind Hermite and cubic spline methods. Finally, the integral operation performed at the end has also been beneficial in this regard.

REFERENCES:

hermite_interpolation(x) function is from on chatgpt(only writing long array) [14] : using method of cubic_spline_interpolation() is from https://www.youtube.com/watch? v=ZhljXJG1Ylg(tridiagonal matrix) cubic formula(x) and derivative_cubic(x) is from on chatgpt (only writing long array) using method of simpson rule and trapezoidal method is from https://www.youtube.com/watch?v=4XdVlALIEkk

In [ ]:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js